

11주차 임베딩, 순환/합성곱 신경망

1. Word2Vec

분포 가설(Distributional Hypothesis) 기반 임베딩 모델 (2013년, 구글)

- **분포 가설**: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정
- 분포 가설은 단어 간의 **동시 발생(co-occurrence)** 확률 분포를 이용해 단어 간의 유사성을 측정
- 예: '내일 자동차를 타고 부산에 간다'와 '내일 비행기를 타고 부산에 간다'에서 '자동차'와 '비행기'는 주변에 분포한 단어들이 동일하거나 유사하므로 비슷한 의미를 가질 것이라고 예상

분산 표현 (Distributed Representation)

- 이러한 가정을 통해 단어의 **분산 표현(Distributed Representation)** 을 학습할 수 있음
- 분산 표현이란 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것을 의미
- 분포 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남
- 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 됨
- 예: '자동차'와 '비행기'라는 단어는 벡터 공간에서 서로 가까운 위치에 표현됨
- 이러한 방법으로 빈도 기반 벡터화 기법에서 발생했던 **단어의 의미 정보를 저장하지 못하는 한계를 극복**
- 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보를 효과적으로 표현
- 이러한 분산 표현 방식은 다양한 자연어 처리 작업에서 높은 성능을 보여주며, 다운스트림 작업에서 더 뛰어난 성능을 보임

희소 표현 vs 밀집 표현

단어를 벡터화하는 방법은 크게 **희소 표현(sparse representation)** 과 **밀집 표현(dense representation)** 으로 나눌 수 있음

- 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법은 희소 표현

- Word2Vec은 밀집 표현

표 6.3 단어의 희소 표현

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814

06 _ 임베딩

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

희소 표현의 문제점

- 원-핫 인코딩과 TF-IDF와 같은 방법은 대부분의 벡터 요소가 0으로 표현되는 희소 표현 방법
- 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 **공간적 낭비** 발생
- 단어 간의 유사성을 반영하지 못하고, 벡터 간의 유사성을 계산하는 데도 많은 비용이 발생

밀집 표현의 장점

- Word2Vec과 같은 밀집 표현은 단어를 고정된 크기의 실수 벡터로 표현하기 때문에 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않음
- 벡터 공간상에서 단어 간의 거리를 효율적으로 계산할 수 있으며, 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간을 활용할 수 있음
- 밀집 표현 벡터화는 학습을 통해 단어를 벡터화하기 때문에 단어의 의미를 비교할 수 있음

- 밀집 표현된 벡터를 **단어 임베딩 벡터(Word Embedding Vector)** 라고 하며, Word2Vec은 대표적인 단어 임베딩 기법 중 하나
- Word2Vec은 밀집 표현을 위해 **CBoW**와 **Skip-gram** 이라는 두 가지 방법을 사용

CBoW (Continuous Bag of Words)

CBoW란 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법

- **중심 단어(Center Word):** 예측해야 할 단어
- **주변 단어(Context Word):** 예측에 사용되는 단어들
- **윈도우(Window):** 중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 정하는 범위
- 이 윈도우를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습
- 학습을 위해 윈도우를 이동해 가며 학습하는데, 이러한 방법을 **슬라이딩 윈도우(Sliding Window)** 라 함
- CBoW는 슬라이딩 윈도우를 사용해 **한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어**를 학습할 수 있음
- 학습 데이터는 **(주변 단어 | 중심 단어)** 로 구성됨

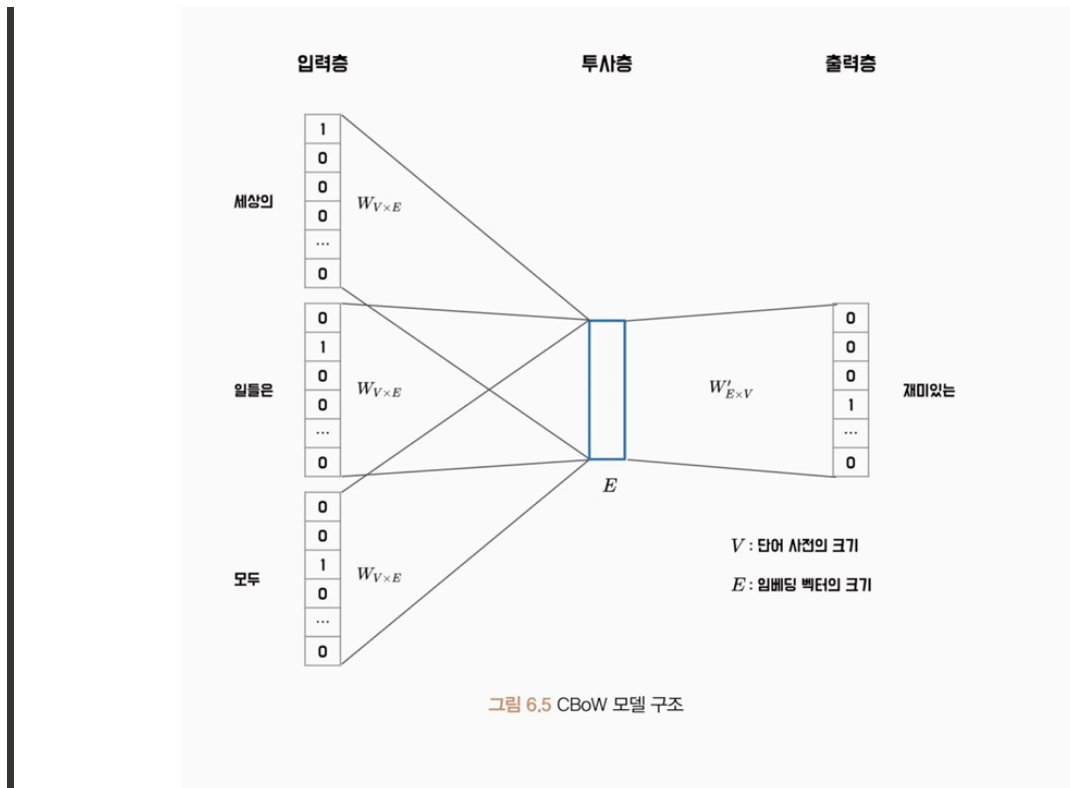
입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

CBoW 모델 구조

- CBoW 모델은 각 입력 단어의 원-핫 벡터를 입력값으로 받음
- 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측
- 입력 단어는 원-핫 벡터로 표현돼 **투사층(Projection Layer)** 에 입력됨

- 투사층이란 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 **순람표(Lookup table, LUT)** 구조가 됨
- 투사층을 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환됨
- 계산된 평균 벡터를 가중치 행렬 $W'_{E \times V}$ 와 곱하면 V 크기의 벡터를 얻음. 벡터에 소프트맥스 함수를 이용해 중심 단어를 예측



Skip-gram

Skip-gram은 CBoW와 반대로 **중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델**

- Skip-gram은 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만들
- 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습시킴
- CBoW는 하나의 윈도우에서 하나의 학습 데이터가 만들어지는 반면, **Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어짐**

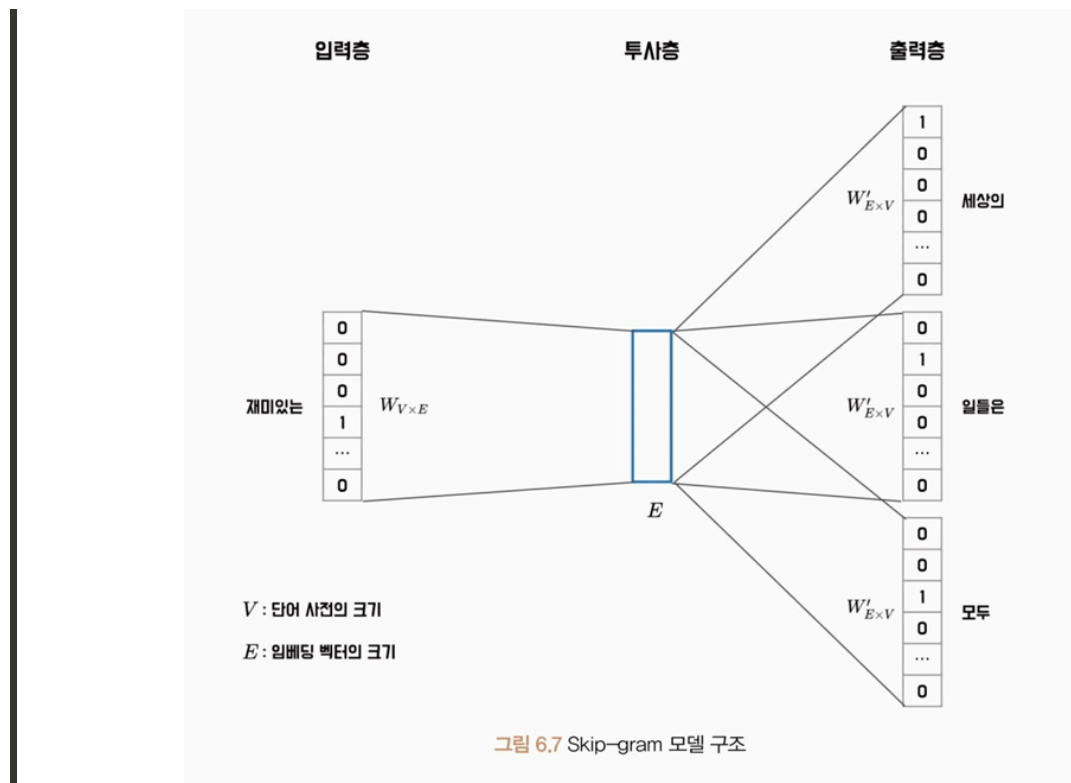
Skip-gram의 장점

- 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하므로 **더 많은 학습 데이터셋을 추출할 수 있으며**, 일반적으로 CBoW보다 더 뛰어난 성능을 보임

- 비교적 드물게 등장하는 단어들을 더 잘 학습할 수 있게 되고 단어 벡터 공간에서 더 유의미한 거리 관계를 형성할 수 있음

Skip-gram 모델 동작

- Skip-gram 모델도 CBoW와 마찬가지로 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져옴
 - 입력 단어의 임베딩과 $W'_{E \times V}$ 가중치와의 곱셈을 통해 V 크기의 벡터를 얻고, 이 벡터에 소프트맥스 연산을 취함으로써 주변 단어를 예측
 - 소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행. 말뭉치의 크기가 커지면 필연적으로 단어 사전의 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 학습 속도가 느려지는 단점이 있음
 - 이를 해결하기 위해 **계층적 소프트맥스**와 **네거티브 샘플링 기법**을 적용해 학습 속도가 느려지는 문제를 완화



학습 속도 개선 기법

계층적 소프트맥스 (Hierarchical Softmax)

- 계층적 소프트맥스(Hierarchical Softmax) 는 출력층을 이진 트리(Binary tree) 구조로 표현해 연산을 수행

- 이때 자주 등장하는 단어일수록 트리의 상위 노드에 위치하고, 드물게 등장하는 단어일수록 하위 노드에 배치됨
- 이러한 방식으로 확률을 계산하면 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성을 보임
- 각 노드는 학습이 가능한 벡터를 가지며, 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률을 계산
- **잎 노드(Leaf Node)** 는 가장 깊은 노드로, 각 단어를 의미하며, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
- 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있음
- 예: '추천해요' 단어는 $0.43 \times 0.74 \times 0.27 = 0.085914$ 의 확률을 갖게 됨. 이 경우 학습 시 1, 2번 노드의 벡터만 최적화하면 됨
- 단어 사전의 크기를 V 라고 했을 때 일반적인 소프트맥스 연산은 $O(V)$ 의 시간 복잡도를 갖지만, 계층적 소프트맥스의 시간 복잡도는 $O(\log_2 V)$ 의 시간 복잡도를 가짐

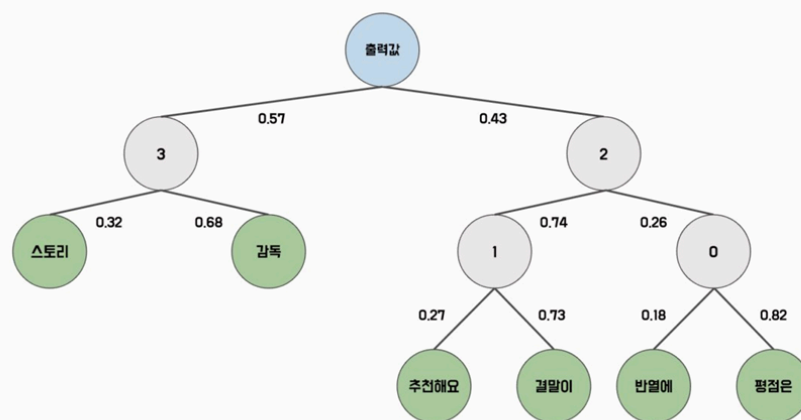


그림 6.8 계층적 소프트맥스 구조

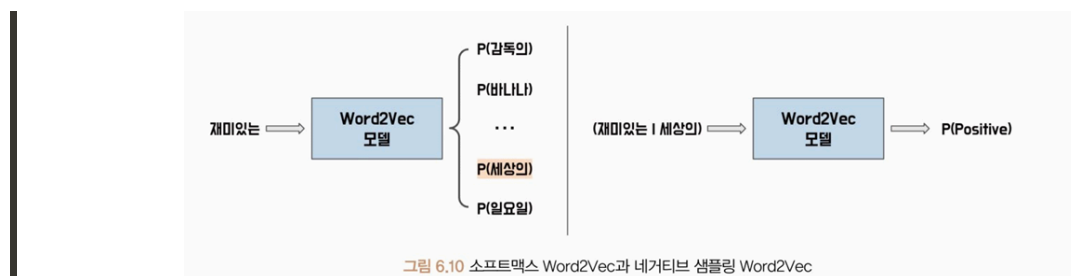
네거티브 샘플링 (Negative Sampling)

- **네거티브 샘플링(Negative Sampling)** 은 Word2Vec 모델에서 사용되는 확률적인 샘플링 기법으로 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용
- 학습 윈도우 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행
- 이를 통해 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습할 수 있음
- 이때 추출할 단어 n 은 일반적으로 **5~20개**를 사용하며, 각 단어가 추출될 확률은 다음과 같이 계산

추출 확률 공식:

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- 네거티브 샘플링 추출 확률을 계산하기 위해 먼저 각 단어 w_i 의 출현 빈도수를 $f(w_i)$ 로 나타냄
- 예: 말뭉치 내에 단어 '추천해요'가 100번 등장했고, 전체 단어의 빈도가 2,000이라면 $f(\text{추천해요}) = \frac{100}{2000} = 0.05$ 가 됨
- $P(w_i)$ 는 단어 w_i 가 네거티브 샘플로 추출될 확률. 이때 출현 빈도수에 **0.75제곱한 값을 정규화 상수로** 사용하는데, 이 값은 실험을 통해 얻어진 최적의 값
- 네거티브 샘플링에서는 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 아니면 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류를 함
- 이를 위해 **로지스틱 회귀 모델**을 사용하며, 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습
- 네거티브 샘플링 모델에서는 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블(1 또는 0)을 가져와 내적 연산을 수행
- 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환됨
- 이때 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화됨
- 즉, **다중 분류에서 이진 분류로 학습 목적이 바뀌게 됨**



단어 유사도: 코사인 유사도 (Cosine Similarity)

$$\text{cosine similarity}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- 임베딩의 유사도를 측정할 때는 **코사인 유사도(Cosine Similarity)**가 가장 일반적으로 사용되는 방법
- 코사인 유사도는 두 벡터 간의 각도를 이용하여 유사도를 계산하며, 두 벡터가 유사할수록 값이 1에 가까워지고, 다를수록 0에 가까워짐
- 두 벡터 간의 코사인 유사도는 두 벡터의 내적을 벡터의 크기(유클리드 노름)의 곱으로 나누어 계산할 수 있음
- a와 b는 유사도를 계산하려는 벡터이며, 두 벡터를 내적한 벡터의 크기의 곱을 나눠 코사인 유사도를 계산할 수 있음. 벡터의 크기는 각 성분의 제곱합에 루트를 씌운 값
- 코사인 유사도는 임베딩 공간에서 단어 간의 유사도를 측정하는 데 매우 유용

`top_n_index` 함수

- 유사도 행렬을 내림차순으로 정렬하여 어떤 단어가 가장 가까운 단어인지 반환
- 입력 단어도 단어 사전에 포함되므로 **입력 단어 자신이 가장 가까운 단어**가 됨. 그러므로 두 번째 가까운 단어부터 계산해 반환

Word2Vec 실습 (Skip-gram, PyTorch)

모델 실습: Skip-gram

Word2Vec 모델은 학습할 단어의 수를 V 로, 임베딩 차원을 E 로 설정해 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬을 최적화하며 학습

- $W_{V \times E}$ 행렬은 **룩업(Lookup)** 연산을 수행하는데, **임베딩(Embedding)** 클래스를 사용하면 간편하게 구현할 수 있음
- 임베딩 클래스는 단어나 범주형 변수와 같은 이산 변수를 연속적인 벡터 형태로 변환해 사용할 수 있음
- 연속적인 벡터 표현은 모델이 학습하는 동안 단어의 의미와 관련된 정보를 포착하고, 이를 기반으로 단어 간의 유사도를 계산
- 이때 이산 변수의 값을 연속적인 벡터로 변환하는 과정을 **룩업**이라 함

임베딩 클래스 파라미터

파라미터	설명
<code>num_embeddings</code>	이산 변수의 개수로 단어 사전의 크기를 의미
<code>embedding_dim</code>	임베딩 벡터의 차원 수로 임베딩 벡터의 크기를 의미
<code>padding_idx</code>	패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정. 병렬 처리를 위해 입력 배치의 문장 길이가 동일해야 하므로 입력 문장들을 일정한 길이로 맞추는 역할. 패딩 인덱스는 임베딩

파라미터	설명
	수보다 작아야 하며 패딩 인덱스의 벡터값은 모델 학습 시 최적화되지 않음
<code>max_norm</code>	임베딩 벡터의 최대 크기를 지정. 각 임베딩 벡터의 크기가 최대 노름 값 이상이면 임베딩 벡터를 최대 노름 크기로 잘라내고 크기를 감소시킴
<code>norm_type</code>	임베딩 벡터의 크기를 제한하는 방법을 선택. 기본값은 2로, 이는 L2 정규화 방식을 사용한다는 의미. 만약 노름 타입을 1로 설정하면 L1 정규화 방식을 사용

전체 실습 흐름

1. 데이터셋 로드 및 토큰화

- 코포라(Korpora) 라이브러리의 네이버 영화 리뷰(NSMC) 테스트 데이터셋 사용
- Okt 토큰라이저로 형태소 추출

2. 단어 사전 구축 (`build_vocab`)

- `Counter` 로 빈도 계산
- `n_vocab` 개의 가장 많이 등장한 토큰으로 사전 구성
- `<unk>` 토큰: OOV(Out Of Vocabulary) 대응용 특수 토큰으로 단어 사전 내에 없는 모든 단어는 `<unk>` 토큰으로 대체됨
- `n_vocab` 매개변수는 구축할 단어 사전의 크기를 의미. 만약 문서 내에 `n_vocab`보다 많은 종류의 토큰이 있다면, 가장 많이 등장한 토큰 순서로 사전을 구축
- 예제는 단어 사전의 최대 길이를 5000으로 입력. 특수 토큰(Special Token)은 현재 1개이므로 단어 사전은 5,001개로 구성됨

3. 단어 쌍 추출 (`get_word_pairs`)

- `window_size` : 주변 단어를 몇 개까지 고려할 것인지를 설정
- `idx` : 현재 단어의 인덱스, `center_word` : 중심 단어
- `window_start` 와 `window_end` : 현재 단어에서 얼마나 멀리 떨어진 주변 단어를 고려할 것인지를 결정. `window_start` 와 `window_end` 는 문장의 경계를 넘어가는 경우가 없게 조정
- 출력: [중심 단어, 주변 단어] 쌍의 리스트

4. 인덱스 쌍 변환 (`get_index_pairs`)

- 단어 쌍을 토큰 인덱스 쌍으로 변환

- 딕셔너리의 `get` 메서드로 토큰이 단어 사전 내에 있으면 해당 토큰의 인덱스를 반환하고, 단어 사전 내에 없다면 `<unk>` 토큰의 인덱스를 반환

5. 데이터로더 적용

- 인덱스 쌍을 텐서 형식으로 변환 → `[N, 2]` 구조
- `TensorDataset`, `DataLoader` 적용 (`batch_size=32`, `shuffle=True`)

6. 모델 정의 (`VanillaSkipgram`)

- `nn.Embedding`: 룩업 계층 구현
- `nn.Linear`: 가중치 행렬 구현
- 단순히 입력 단어와 주변 단어를 룩업 테이블에서 가져와서 내적을 계산한 다음, 손실 함수를 통해 예측 오차를 최소화하는 방식으로 학습됨
- 손실 함수: `CrossEntropyLoss` (내부적으로 소프트맥스 연산을 수행하므로 신경망의 출력값을 후처리 없이 활용할 수 있음)
- 최적화: `SGD` (`lr=0.1`)

7. 모델 학습 (10 에폭)

- 모델 학습 구조는 1부에서 다룬 구조와 동일
- 모델 학습이 완료되면 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬 중 하나의 행렬을 선택해 임베딩 값을 추출

8. 임베딩 값 추출 및 유사도 계산

- Word2Vec 모델의 임베딩 행렬을 이용해 각 단어의 임베딩 값을 매핑
- `cosine_similarity` 함수: a 매개변수는 임베딩 토큰을 의미하며, b 매개변수는 임베딩 행렬을 의미. 임베딩 행렬은 `[5001, 128]`의 구조를 가지므로 노름을 계산할 때 `axis=1` 방향으로 계산
- `top_n_index` 함수: 유사도 행렬을 내림차순으로 정렬해 어떤 단어가 가장 가까운 단어인지 반환. 입력 단어 자신이 가장 가까운 단어가 되므로 두 번째 가까운 단어부터 계산해 반환
- 예: 입력된 단어가 '연기'일 때 가장 유사한 단어 3개로 '연기력', '배우', '시나리오'가 추출된 것을 확인할 수 있음

모델 실습: Gensim

- 매우 간단한 구조의 Word2Vec 모델을 학습할 때 데이터 수가 적은 경우에도 학습하는데 오랜 시간이 소요됨

- 이러한 경우, 계층적 소프트맥스나 네거티브 샘플링 같은 기법을 사용하면 더 효율적으로 학습할 수 있음
- **젠심(Gensim)** 라이브러리를 활용하면 Word2Vec과 같은 자연어 처리 모델을 쉽게 구성할 수 있음
- 젠심 라이브러리는 대용량 텍스트 데이터의 처리를 위한 **메모리 효율적인 방법**을 제공해 대규모 데이터셋에서도 효과적으로 모델을 학습할 수 있음
- 또한 학습된 모델을 저장하여 관리할 수 있고, 비슷한 단어 찾기 등 유사도와 관련된 기능도 제공하여 자연어 처리에 필요한 다양한 기능을 제공
- 젠심 라이브러리는 **사이썬(Cython)** 을 이용해 병렬 처리나 네거티브 샘플링 등을 적용. 사이썬은 C++ 기반의 확장 모듈을 파이썬 모듈로 컴파일하는 기능을 제공. 젠심으로 모델을 구성한다면 파이토치를 이용한 학습보다 훨씬 더 빠른 속도로 학습할 수 있음
- 젠심 라이브러리의 Word2Vec 클래스는 네거티브 샘플링 등에 필요한 여러 하이퍼파라미터를 받아 쉽고 빠르게 모델을 학습할 수 있음

Word2Vec 클래스 주요 파라미터

파라미터	설명
<code>sentences</code>	모델의 학습 데이터를 나타내며 토큰 리스트로 표현됨. 이렇게 구성된 학습 데이터는 학습 문장들의 리스트로 이루어짐
<code>corpus_file</code>	학습 데이터를 파일로 입력할 때 파일의 경로를 의미. 입력 문장 대신에 사용할 수 있음
<code>vector_size</code>	학습할 임베딩 벡터의 크기를 의미하며, 임베딩 차원 수를 설정
<code>alpha</code>	Word2Vec 모델의 학습률을 의미
<code>window</code>	학습 데이터를 생성할 윈도우의 크기를 의미. 예를 들어 윈도우가 3이라면 중심 단어로부터 3만큼 떨어진 단어까지 주변 단어로 고려해 데이터를 생성
<code>min_count</code>	학습에 사용할 단어의 최소 빈도를 의미. 말뭉치 내에 최소 빈도만큼 등장하지 않은 단어는 학습에 사용되지 않음
<code>max_final_vocab</code>	단어 사전의 최대 크기를 의미. 최소 빈도를 충족하는 단어가 최대 최종 단어 사전보다 많으면 자주 등장한 단어 순으로 단어 사전을 구축
<code>workers</code>	빠른 학습을 위해 병렬 학습할 스레드의 수를 의미
<code>sg</code>	Skip-gram 모델을 사용 여부를 설정. 1이라면 Skip-gram을, 0이라면 CBoW 모델을 사용

파라미터	설명
<code>hs</code>	계층적 소프트맥스 사용 여부를 설정. 1이라면 계층적 소프트맥스를 사용하고, 0이라면 사용하지 않음
<code>cbow_mean</code>	CBoW 모델로 구성할 때 사용되는 하이퍼파라미터로 중심 단어와 주변 단어를 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부를 설정. 1이라면 평균화하며, 0이라면 평균화하지 않고 합산
<code>negative</code>	네거티브 샘플링의 단어 수를 의미. 0이라면 네거티브 샘플링을 사용하지 않음
<code>ns_exponent</code>	네거티브 샘플링 확률의 지수를 의미. 수식 6.9는 네거티브 지수가 0.75일 때의 확률 계산식
<code>epochs</code>	학습 에폭 수를 의미
<code>batch_words</code>	몇 개의 단어로 학습 배치를 구성할지 결정. 컴퓨팅 자원이 모자란다면 배치 단어 수를 낮추어 학습을 진행할 수 있음

- `wv` 속성: **Word2VecKeyedVectors** 객체를 반환. 이 객체는 단어 벡터 검색과 유사도 계산 등의 작업을 수행하는 데 사용됨
 - `most_similar()` : 주어진 단어에 대해 가장 유사한 단어를 찾는 메서드
 - `similarity()` : 두 단어 간의 유사도를 계산하는 메서드
- 파이토치 모델 파일과 구분하기 위해 `.model` 과 같은 형식으로 저장

2. fastText

(2015년, 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델)

- 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘
- fastText는 단어와 문장을 벡터로 변환하는 기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용됨
- 이러한 벡터화 기술은 Word2Vec과 유사하지만, **fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능을 제공**
- 하위 단어를 고려하기 위해 **N-gram**을 사용해 단어를 분해하고 벡터화하는 방법으로 동작

Word2Vec의 한계

- 이번 절에서 다룬 Word2Vec은 분포 가설을 통해 쉽고 빠르게 단어의 임베딩을 학습할 수 있지만, 이는 **단어의 형태학적 특징을 반영하지 못한다는 한계**가 있음
- 예를 들어 교착어로서 한국어는 어근과 접사, 조사 등으로 이루어지는 규칙을 가지고 있기

때문에 Word2Vec 모델에서는 이러한 구조적 특징을 제대로 학습하기가 어려움

- 이러한 한계는 **제한된 단어 사전에서 많은 OOV를 발생**시키는 원인이 됨

fastText의 동작 원리

- fastText에서는 단어의 벡터화를 위해 **<**, **>** 와 같은 특수 기호를 사용하여 단어의 시작과 끝을 나타냄
- 이러한 기호는 단어의 하위 문자열을 고려하는 데 중요한 역할을 함
- 기호가 추가된 단어는 N-gram을 사용하여 **하위 단어 집합(Subword set)** 으로 분해됨
- 예: '서울특별시'를 바이그램으로 분해하면 '<서울', '울특', '특별', '별시>'가 됨
- 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함되며, 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 됨
- 이러한 하위 단어 벡터들은 단어의 벡터 표현을 구성하며, 이를 사용하여 자연어 처리 작업을 수행



fastText가 입력으로 받은 '서울특별시'라는 토큰에 대해 처리하는 단계

1. 토큰의 양 끝에 '<'와'>'를 붙여 토큰의 시작과 끝을 인식할 수 있게 함
 2. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해. 트라이그램을 사용한다면, '서울특별시'는 [<서울, 서울특, 울특별, 특별시, 별시>]로 분해됨
 3. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함. 이렇게 하위 단어 집합이 만들어지면, 각 하위 단어는 고유한 벡터값을 갖게 됨
- fastText는 위와 같은 방법으로 하위 단어 집합을 생성. 입력 단어를 하위 단어로 분해하면 총 6개의 하위 단어가 존재
 - 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산
 - 일반적으로 fastText는 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 단어 벡터를 구성함으로써 단어의 부분 문자열을 고려하는 유연하고 정확한 하위 단어 집합을

생성

fastText의 장점

- 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습할 수 있음 → 비슷한 단어끼리는 비슷한 임베딩 벡터를 갖게 되어, 단어 간 유사도를 높일 수 있음
- **OOV 단어도 하위 단어로 나누어 임베딩을 계산할 수 있게 됨**
 - 하위 단어 기반의 임베딩 방법을 사용하면, 말뭉치에 등장하지 않은 단어라도 유사한 하위 단어를 가지고 있으면 유사한 임베딩 벡터를 갖게 됨
 - 한국어와 같은 많은 언어는 형태적 구조를 갖고 있기 때문에 하위 단어로 나누어 임베딩을 학습하는 fastText의 접근 방식을 통해 OOV 문제를 해결할 수 있음

fastText 실습 (Gensim)

- fastText와 Word2Vec 모델 모두 단어 임베딩을 학습하는 데 사용되는 알고리즘
- 두 알고리즘 모두 단어를 고정 길이 벡터 형태로 표현하기 위해 분산 표현 학습을 수행하고 주변 단어의 정보를 활용하여 단어의 의미를 파악
- fastText 모델도 CBoW와 Skip-gram으로 구성되며 네거티브 샘플링 기법을 사용해 학습
- **Word2Vec 모델은 기본 단위로 모델을 학습했다면 fastText는 하위 단어로 학습** → Word2Vec 모델보다 입력 단어를 구성하는 데 조금 더 복잡한 과정이 필요하고 모든 하위 단어 크기를 갖는 임베딩 계층이 필요
- 젠심 라이브러리는 이러한 복잡한 과정을 **FastText 클래스**로 제공. Word2Vec과 기술적인 공통점이 많으므로 Word2Vec의 대부분 매개변수를 공유

데이터셋: KorNLI (Korean Natural Language Inference)

- 한국어 **자연어 추론(Natural Language Inference, NLI)** 을 위한 데이터셋
- 자연어 추론이란 두 개 이상의 문장이 주어졌을 때, 두 문장 간의 관계를 분류하는 작업을 의미
- 주어진 문장이 **함의 관계(entailment)**, **중립 관계(neutral)**, **불일치 관계(contradiction)** 중 어느 관계에 해당하는지 분류할 수 있음
- 코포라 라이브러리의 KorNLI 인스턴스는 `get_all_texts` 와 `get_all_pairs` 메서드를 제공
- `get_all_texts` 메서드: 모든 문장을 튜플 형태로 반환
- `get_all_pairs` 메서드: 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환

- 단어 임베딩 모델을 학습하는 것이 목적이므로 두 문장의 관계를 고려하지 않고 입력 문장과 대응 문장 전체를 이용해 학습을 진행
- Word2Vec 모델과 다르게, fastText 모델은 입력 단어의 구조적 특징을 학습할 수 있음 → 형태소 분석기를 통해 토큰화하지 않고 띄어쓰기를 기준으로 단어를 토큰화해 학습을 진행

FastText 클래스 추가 파라미터 (Word2Vec과의 차이)

파라미터	설명
<code>min_n</code>	사용할 N-gram의 최소값을 의미
<code>max_n</code>	사용할 N-gram의 최대값을 의미

- 대부분의 하이퍼파라미터는 Word2Vec 클래스의 하이퍼파라미터와 동일하지만, N-gram 범위를 결정하는 하이퍼파라미터가 추가됨
- 가령 최소 N이 2이고 최대 N이 4라면 입력 단어를 2-gram, 3-gram, 4-gram으로 나누어 하위 단어 집합을 생성

OOV 처리 실습

- Word2Vec 모델은 단어 사전에 존재하지 않는 단어는 임베딩을 계산할 수 없었음
- fastText는 하위 단어로 나뉘어 있기 때문에 OOV 단어도 처리할 수 있음

3. 순환 신경망 (RNN)

자연어와 연속형 데이터

순환 신경망(Recurrent Neural Network, RNN) 모델은 순서가 있는 연속적인 데이터 (Sequence data) 를 처리하는 데 적합한 구조를 갖고 있음

- 순환 신경망은 각 **시점(Time step)** 의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동
- 예: 일일 주가 데이터셋이 있다고 가정한다면 3월 7일의 주가는 3월 6일 주가의 영향을 받았을 가능성이 높음
- 이렇게 특정 시점 t 에서의 데이터가 이전 시점($t_0, t_1, \dots, t_{n-1}$)의 영향을 받는 데이터를 연속형 데이터라 함

자연어 데이터의 연속형 특성

- 자연어 데이터는 연속적인 데이터의 일종. 자연어 데이터는 문장 안에서 단어들이 순서대로 등장하므로, 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장의 의미를 형성

- 연속형 데이터의 특성 예시:
 - '금요일이 지나면 _': 이 문장을 보고 '주말' 또는 '토요일' 등의 단어가 등장할 것이라고 예측할 수 있음
 - '수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면 _': 앞선 단어 패턴을 고려했을 때 '주말'보다는 '**토요일**' 이 더 자연스러운 예측일 것
- 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미를 형성. 이러한 특징으로 인해 자연어는 연속형 데이터의 특성을 가짐
- t 번째 단어는 $t-1$ 번째까지의 단어에 영향을 받아 결정
- 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 **상관관계(Correlation)** 가 존재

순환 신경망

- 순환 신경망은 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류
- 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용됨
- 순환 신경망은 **시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터**와 같은 도메인에서 널리 사용됨
- 이러한 데이터는 일반적으로 길이가 가변적이며, 순서에 따라 의미가 있기 때문에, 순환 신경망은 이러한 데이터를 처리하기에 적합한 구조를 가지고 있음
- 순환 신경망은 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 **은닉 상태 (Hidden state)** 의 형태로 저장
- 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드를 순환 신경망의 **셀(Cell)** 이라 함
- 순환 신경망의 셀은 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉 상태 h_t 를 계산. 이러한 재귀적 특징으로 인해 '순환' 신경망이라 불림

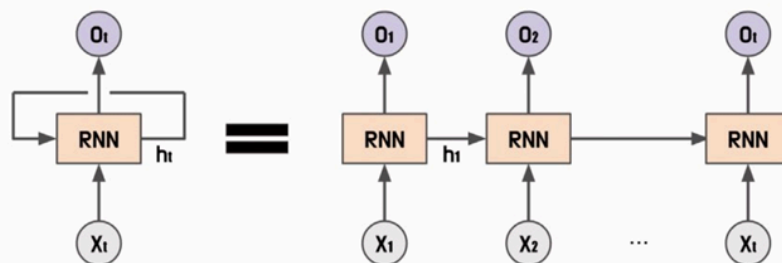


그림 6.12 순환 신경망의 셀

핵심 수식

순환 신경망은 각 시점 t 에서 현재 입력값 x_t 와 이전 시점 $t-1$ 의 은닉 상태 h_{t-1} 를 이용해 현재 시점의 은닉 상태 h_t 와 출력값 y_t 를 계산

- **은닉 상태:** $h_t = \sigma_h(h_{t-1}, x_t) = \sigma_h(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$
 - σ_h 는 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
 - 은닉 상태 활성화 함수는 이전 시점 $t-1$ 의 은닉 상태 h_{t-1} 과 입력값 x_t 를 입력받아 현재 시점의 은닉 상태 h_t 를 계산
 - σ_h 는 가중치(W)와 편향(b)을 이용해 계산
 - W_{hh} 는 이전 시점의 은닉 상태 h_{t-1} 에 대한 가중치, W_{hx} 는 입력값 x_t 에 대한 가중치, b_h 는 은닉 상태 h_t 의 편향을 의미
- **출력값:** $y_t = \sigma_y(h_t) = \sigma_y(W_{hy}h_t + b_y)$
 - σ_y 는 순환 신경망의 출력값을 계산하기 위한 활성화 함수
 - 출력값 활성화 함수는 현재 시점의 은닉 상태 h_t 를 입력으로 받아 출력값 y_t 를 계산
 - W_{hy} 는 현재 시점의 은닉 상태 h_t 에 대한 가중치, b_y 는 출력값 y_t 의 편향을 의미
 - 출력값 계산 방법도 가중치(W)와 편향(b)을 이용해 계산
- 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터를 처리할 수 있음

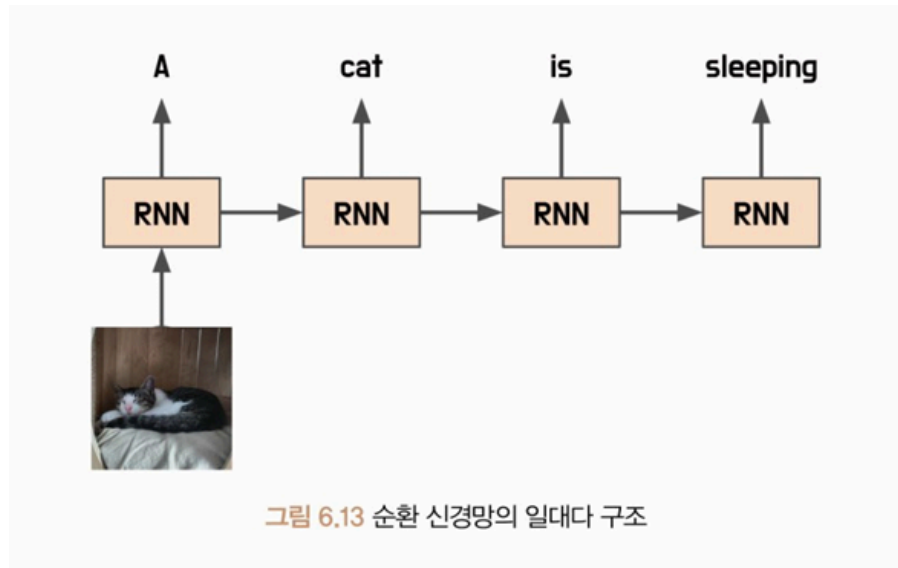
순환 신경망 구조 유형

순환 신경망은 다양한 구조로 모델을 설계할 수 있음. 가장 단순한 구조인 단순 순환 구조부터 일대다 구조, 다대일 구조, 다대다 구조 등이 있음

일대다 구조 (One-to-Many)

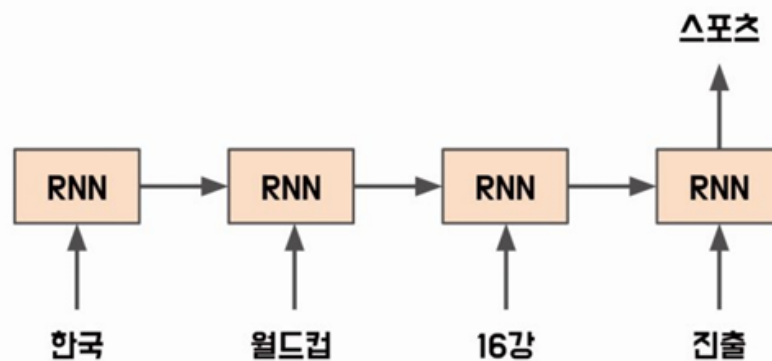
- **일대다 구조(One-to-Many)**는 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조
- 예: 자연어 처리 분야에서는 일대다 구조를 사용하여 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측하는 작업을 할 수 있음. 입력 시퀀스는 문장으로 이루어져 있으며, 출력 시퀀스는 각 단어의 품사로 이루어져 있음
- 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 **이미지 캡셔닝 (Image Captioning)** 모델이 됨. 입력 시퀀스는 이미지로 이루어져 있으며, 출력값은 이미지에 대한 설명 문장들로 구성됨

- 이러한 일대다 구조를 구현하기 위해서는 **출력 시퀀스의 길이를 미리 알고 있어야 함**. 이를 위해 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이를 예측하는 모델을 함께 구현해야 함



다대일 구조 (Many-to-One)

- **다대일 구조(Many-to-One)** 는 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조
- 예: **감성 분류(Sentiment Analysis)** 분야에서는 다대일 구조를 사용하여 특정 문장의 감정(긍정, 부정)을 예측하는 작업을 할 수 있음. 이때, 입력 시퀀스는 문장으로 이루어져 있으며, 출력값은 해당 문장의 감정(긍정, 부정)으로 이루어져 있음
- 입력 시퀀스가 어떤 범주에 속하는지를 구분하는 문장 분류, 두 문장 간의 관계를 추론하는 **자연어 추론(Natural Language Inference)** 등에도 적용할 수 있음



다대다 구조 (Many-to-Many)

- **다대다 구조(Many-to-Many)** 는 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조
- 이 구조는 다양한 분야에서 활용되며, 예를 들어 입력 문장에 대해 번역된 출력 문장을 생성하는 번역기, 음성 인식 시스템에서 음성 신호를 입력으로 받아 문장을 출력하는 음성 인식기 등에서 사용됨
- 다대다 구조에서도 입력 시퀀스와 출력 시퀀스의 길이가 서로 다른 경우가 있을 수 있음. 예를 들어, 입력 문장의 길이와 출력 문장의 길이가 일치하지 않는 경우. 이 경우에는 입력 시퀀스와 출력 시퀀스의 길이를 맞추기 위해 **패딩을 추가하거나 잘라내는 등의 전처리 과정**이 수행됨
- 다대다 구조는 **시퀀스-시퀀스(Seq2Seq)** 구조로 이뤄져 있음. 시퀀스-시퀀스 구조는 입력 시퀀스를 처리하는 **인코더(Encoder)** 와 출력 시퀀스를 생성하는 **디코더(Decoder)** 로 구성됨
- 인코더는 입력 시퀀스를 처리해 고정 크기의 벡터를 출력하고, 디코더는 이 벡터를 입력으로 받아 출력 시퀀스를 생성

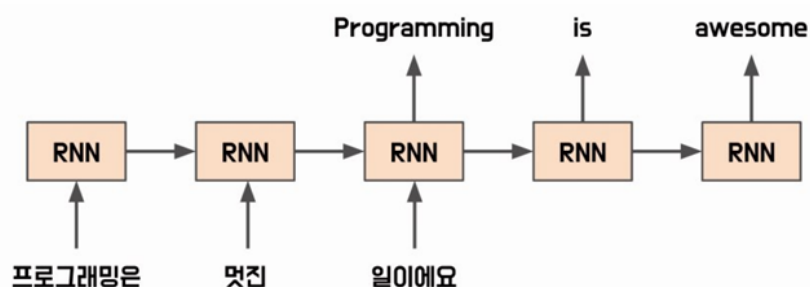
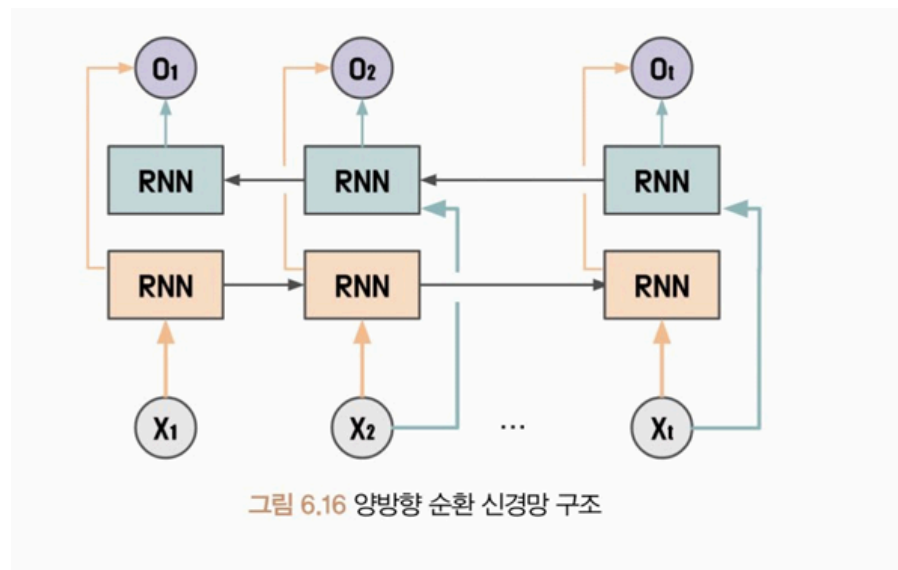


그림 6.15 순환 신경망의 다대다 구조

양방향 순환 신경망 (BiRNN)

- **양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN)** 은 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식
- 순환 신경망은 현재 시점의 입력값을 처리하기 위해 이전 시점($t-1$)의 은닉 상태를 이용하는데, 양방향 순환 신경망에서는 이전 시점($t-1$)의 은닉 상태뿐만 아니라 이후 시점($t+1$)의 은닉 상태도 함께 이용
- 예: "인생은 B와 _ 사이의 C다."라는 문장에서 _에 입력될 단어를 예측해야 한다면 앞의 문장만이 아니라 뒤의 문장에도 영향을 받는다는 것을 알 수 있음. 빈칸 앞의 '인생은 B와'만 보서는 다음에 어떤 단어가 올지 예측하기 어렵지만, 빈칸 뒤의 '사이의 C이다'를 통해 빈칸에 'D'라는 단어가 적절하게 들어갈 수 있다는 것을 알 수 있음

- 이처럼 양방향 순환 신경망은 t 시점 이후의 데이터도 t 시점의 데이터를 예측하는 데 사용될 수 있음
- 이러한 방법은 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로 거꾸로 읽어 들여 처리하는 방식으로 이루어짐
- 대부분의 연속형 데이터는 이전 시점 데이터뿐만 아니라 이후 시점의 데이터와 큰 상관 관계를 갖고 있음. 그러므로 양방향 순환 신경망은 양방향적인 정보를 모두 고려하여 현재 시점의 출력값을 계산



다중 순환 신경망 (Stacked RNN)

- **다중 순환 신경망(Stacked Recurrent Neural Network)** 은 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계돼 있음
- 3.10 '퍼셉트론' 절의 다층 퍼셉트론을 보면 더 복잡한 문제 해결을 위해 단층 퍼셉트론을 여러 개 쌓아 해결했음. 동일한 방식으로 순환 신경망도 여러 층을 쌓아 활용할 수 있음
- 다중 순환 신경망은 여러 개의 순환 신경망 층으로 구성되며, 각 층에서의 출력값은 다음 층으로 전달되어 처리됨
- 이렇게 여러 개의 층으로 구성된 RNN은 데이터의 다양한 특징을 추출할 수 있어 성능이 향상될 수 있음. 또한, 층이 깊어질수록 더 복잡한 패턴을 학습할 수 있다는 장점이 있음
- 하지만 순환 신경망 층이 많아질수록 학습 시간이 오래 걸리고, **기울기 소실 문제**가 발생할 가능성도 높아짐

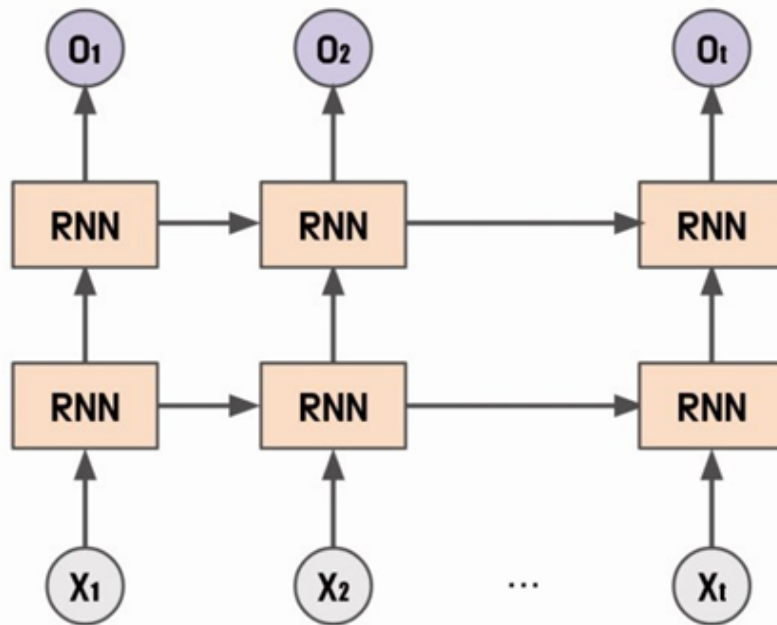


그림 6.17 다중 순환 신경망 구조

순환 신경망 클래스 (PyTorch)

파이토치는 순환 신경망을 쉽게 구현할 수 있도록 순환 신경망 클래스를 제공. 순환 신경망 클래스는 앞서 설명한 일대다일 구조 순환 신경망부터 다중 순환 신경망까지 쉽게 구현할 수 있음

파라미터	설명
<code>input_size</code>	입력 특성 크기(input_size)와 은닉 상태 크기(hidden_size)를 입력해 순환 신경망을 구성할 수 있음. 입력 특성은 입력값 x 를 의미하며, 은닉 상태는 h 를 의미
<code>hidden_size</code>	은닉 상태의 크기
<code>num_layers</code>	순환 신경망의 층수를 의미. 2 이상이면 다중 순환 신경망을 구성
<code>nonlinearity</code>	순환 신경망에서 사용되는 활성화 함수 σ 의 종류를 설정. tanh와 relu를 적용할 수 있음
<code>bias</code>	편향 값 사용 유무를 설정
<code>batch_first</code>	입력 배치 크기를 첫 번째 차원으로 사용할지 여부를 설정. 참 값이라면 [배치 크기, 시퀀스 길이, 입력 특성 크기]의 형태로 입력해야 하며, 거짓 값이라면 [시퀀스 길이, 배치 크기, 입력 특성 크기]로 입력
<code>dropout</code>	과대적합 방지를 위한 드롭아웃 확률을 설정

파라미터	설명
<code>bidirectional</code>	순환 신경망 구조가 양방향으로 처리할지 여부를 설정

입출력 차원

- 순환 신경망 클래스는 순전파 연산 시 입력값(inputs)과 초기 은닉 상태(h_0)로 순방향 연산을 수행해 출력값(outputs)과 최종 은닉 상태(hidden)를 반환
- 입력값 차원은 [배치 크기, 시퀀스 길이, 입력 특성 크기]로 전달
- 초기 은닉 상태는 순방향 계층 수와 양방향 구조에 따라 전달해야 하는 크기가 달라짐: [계층 수 × 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]로 구성
- 순방향 연산 결과는 출력값과 최종 은닉 상태가 반환됨. 출력값은 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) × 은닉 상태 크기]가 됨
- 최종 은닉 상태는 초기 은닉 상태와 동일한 차원으로 반환됨
- 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의

4. LSTM (장단기 메모리)

RNN의 한계

- 일반적인 순환 신경망은 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하는 구조이므로 **시간적으로 먼 과거의 정보는 잘 기억하지 못함**
- 하지만 어떤 연속형 데이터는 다음 시점의 데이터를 유추하기 위해 훨씬 먼 시점의 데이터에서 힌트를 얻어야 함
- 순환 신경망의 경우 시간적으로 연속된 데이터를 다룰 수 있다는 장점이 있지만, **앞선 시점에서의 정보를 끊임없이 반영하기에 학습 데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달되지 않는다는 단점**이 있음
- 이러한 단점으로 인해 **장기 의존성 문제(Long-term dependencies)**가 발생할 수 있음
- 또한, 활성화 함수로 사용되는 하이퍼볼릭 탄젠트 함수나 ReLU 함수의 특성으로 인해 역전파 과정에서 **기울기 소실이나 기울기 폭주**가 발생할 가능성이 있음

LSTM 핵심 구조 (1997, Hochreiter & Schmidhuber)

- 이러한 문제를 해결하기 위해 장단기 메모리를 사용

- 장단기 메모리는 순환 신경망과 비슷한 구조를 가지지만, **메모리 셀(Memory cell)** 과 **게이트(Gate)** 라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제를 해결

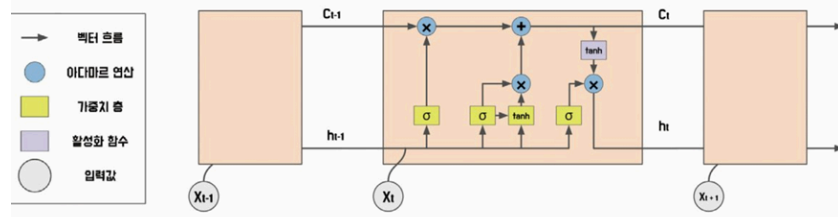


그림 6.18 장단기 메모리 구조

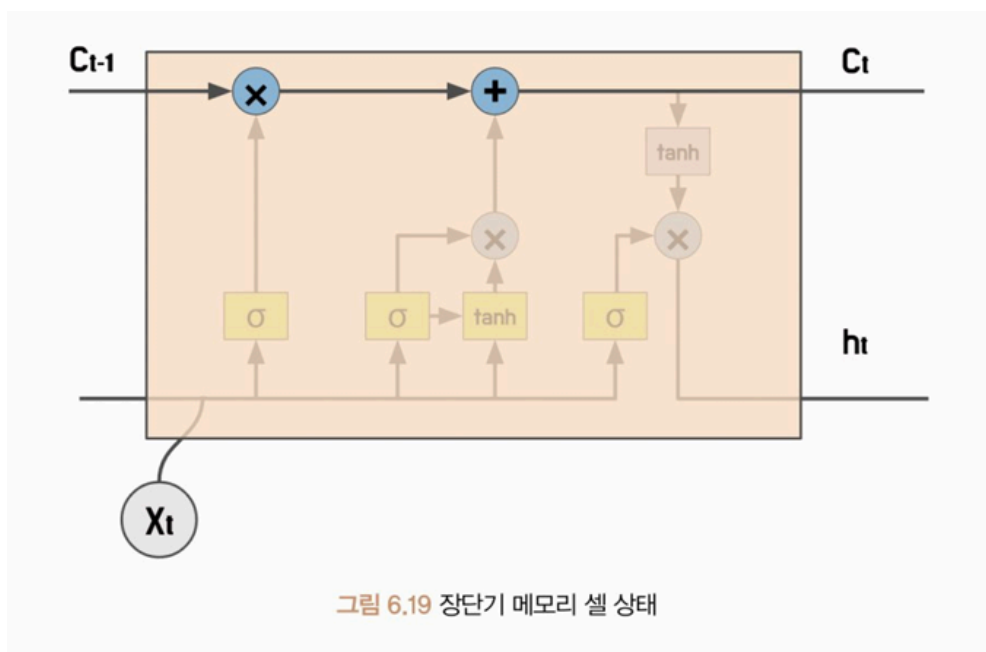
- 장단기 메모리는 **셀 상태(Cell state)** 와 **망각 게이트(Forget gate)**, **기억 게이트(Input gate)**, **출력 게이트(Output gate)** 로 정보의 흐름을 제어
- 셀 상태**: 정보를 저장하고 유지하는 메모리 역할을 하며 출력 게이트와 망각 게이트에 의해 제어됨
- 메모리 셀은 순환 신경망의 은닉 상태와 유사하게 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할을 함
- 하지만 순환 신경망에서 은닉 상태는 출력값을 계산하는 데 사용되지만, 장단기 메모리의 메모리 셀은 출력값 계산에 직접 사용되지 않음
- 대신 장단기 메모리는 망각 게이트, 입력 게이트, 출력 게이트를 통해 어떤 정보를 버릴지, 기억할지, 출력할지를 결정하는 추가적인 연산을 수행
- 세 가지 게이트는 모두 활성화 함수로 **시그모이드**를 사용. 시그모이드 함수는 입력값을 0과 1 사이의 값으로 변환하므로 게이트의 출력값은 각각 0에서 1 사이의 값으로 결정됨. 이 값이 해당 게이트가 입력값에 대해 얼마나 많은 정보를 통과시킬지 결정
- 예: 망각 게이트의 출력값이 1이면 이전 시점의 기억 상태가 현재 시점의 기억 상태에 완전히 유지되며, 0이면 이전 시점의 기억 상태는 현재 시점의 기억 상태에 전혀 반영되지 않음
- 이러한 게이트들은 **입력값에 대한 가중치를 동적으로 조절**하면서 적절한 정보만을 기억하고 유지할 수 있다는 장점을 가지고 있음

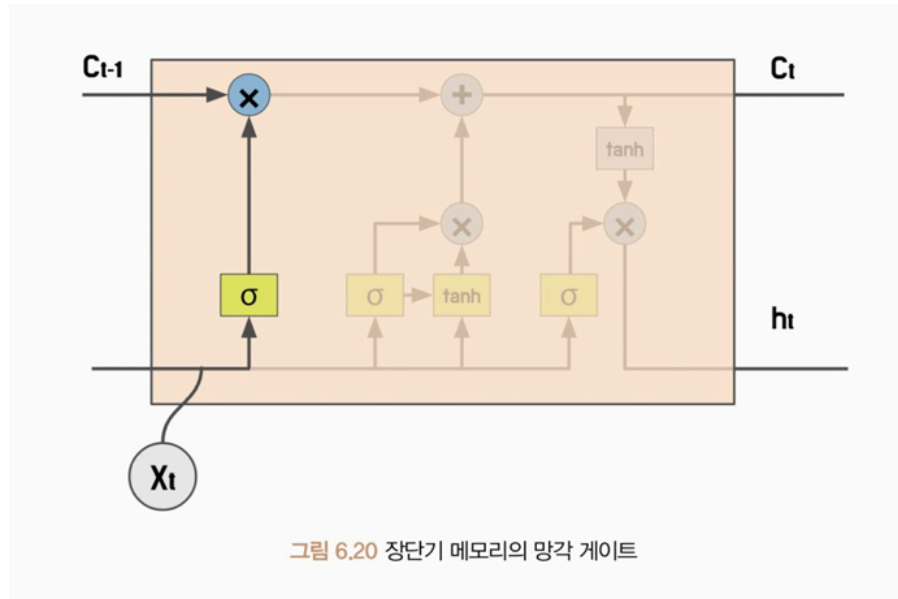
4가지 핵심 구성 요소

구성 요소	역할
셀 상태 (Cell State)	정보를 저장하고 유지하는 메모리 역할. 출력 게이트와 망각 게이트에 의해 제어됨

구성 요소	역할
망각 게이트 (Forget Gate)	장단기 메모리에서 이전 셀 상태에서 어떠한 정보를 삭제할지 결정하는 역할. 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 삭제할지 결정
기억 게이트 (Input Gate)	새로운 정보를 어떤 부분에 추가할지 결정하는 역할. 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정
출력 게이트 (Output Gate)	셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할. 현재 입력과 이전 셀 상태, 그리고 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정

장단기 메모리 구조 상세





망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

- 망각 게이트는 이전 시점의 은닉상태 h_{t-1} 과 현재 시점의 입력값 x_t 를 통해 연산을 수행
- 망각 게이트는 이전 시점의 메모리 셀과 현재 시점의 입력을 바탕으로 어떤 정보를 삭제할지를 결정. 망각 게이트의 출력값 f_t 는 시그모이드 활성화 함수를 사용해 계산됨
- $W_x^{(f)}$ 와 $W_h^{(f)}$ 는 입력값과 은닉 상태를 위한 가중치를 의미하며, $b^{(f)}$ 는 망각 게이트의 편향을 의미
- 현재 시점의 입력값 x_t 와 t-1 시점의 은닉 상태 h_{t-1} 을 사용해 망각 게이트의 출력값을 계산하게 됨
- 망각 게이트는 두 가중치를 통해 망각 게이트 출력값을 최적화. 망각 게이트 출력값은 메모리 셀을 계산하기 위한 가중치로 사용됨
- 망각 게이트의 출력값은 0에서 1 사이의 값으로, 이 값이 1에 가까우면 더 많은 정보를 유지하고 반대로 0에 가까우면 더 많은 정보를 삭제. 출력값이 정확히 1이라면 아무런 정보를 삭제하지 않으며, 0이라면 모든 정보를 삭제
- 따라서 망각 게이트 값 f_t 가 0에 가까울수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향을 미치지 않게 됨

기억 게이트

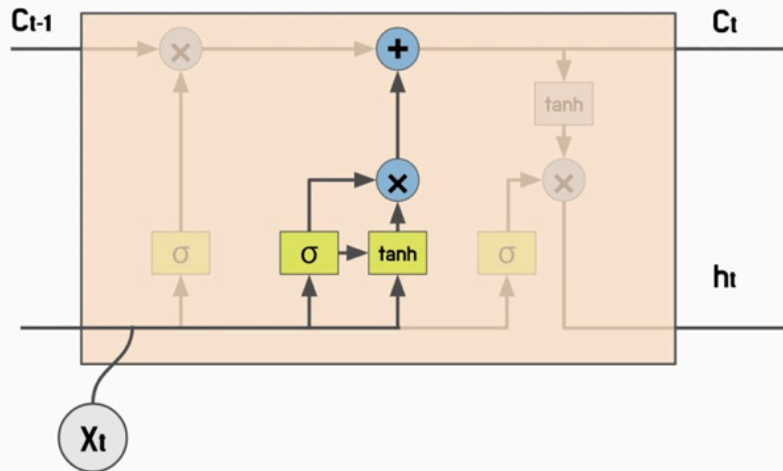


그림 6.21 장단기 메모리의 기억 게이트

$$g_t = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_t = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

- 기억 게이트는 g_t 와 i_t 로 구성돼 있으며 망각 게이트와 동일한 연산으로 계산됨
- g_t 는 활성화 함수로 하이퍼볼릭 탄젠트를 사용하며, i_t 는 시그모이드를 사용
- g_t 는 -1에서 1 사이의 값을 가지므로 이전 시점의 은닉 상태와 현재 시점의 입력값은 모두 $[-1, 1]$ 범위 안에 존재
- g_t 만으로는 새로운 은닉 상태를 계산하기 위해 이 은닉 상태를 얼마나 기억할지 제어하기가 어려움
- 그러므로 i_t 값으로 새로운 은닉 상태의 기억을 제어. i_t 는 $[0, 1]$ 의 값을 가지므로 현재 시점에서 얼마나 많은 정보를 기억할 것인지를 결정하는 가중치 역할을 함. i_t 가 1에 가까울수록 기억할 정보가 많아지고, 0에 가까울수록 정보를 망각하게 됨

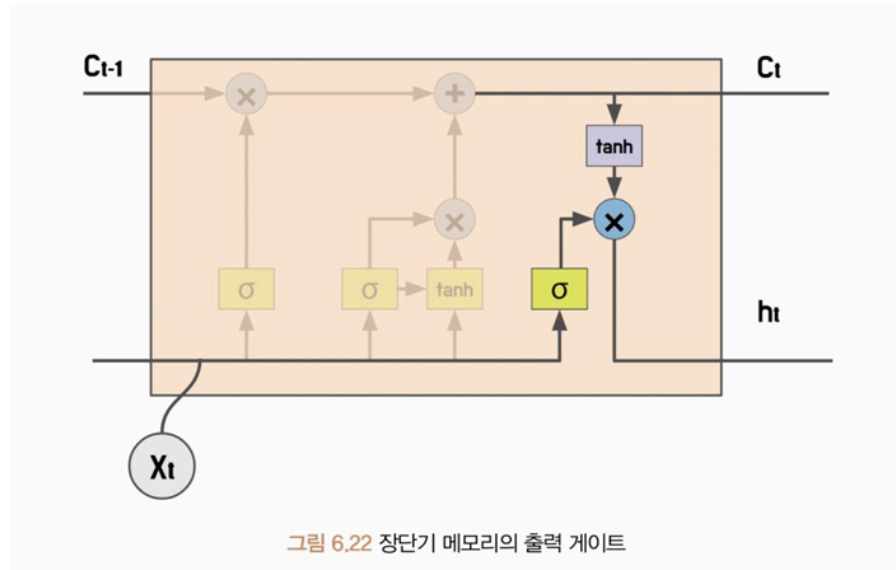
메모리 셀

$$c_t = f_t \odot c_{t-1} + g_t \odot i_t$$

- 메모리 셀은 망각 게이트와 기억 게이트의 정보로 현재 시점의 메모리 셀 값을 계산
- f_t 는 망각 게이트 출력값이며, c_{t-1} 은 이전 시점의 메모리 셀 값을 나타냄. 이 값을 원소별 곱셈 연산을 **아다마르 곱(Hadamard Product, $\odot$)** 하면 현재 시점의 메모리 셀 값이 계산됨

- 따라서 망각 게이트 값 f_t 가 0에 가까울수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향을 미치지 않게 됨
- 기억 게이트도 아다마르 곱을 통해 계산되며 망각 게이트와 합산됨
- 망각 게이트는 이전 시점의 메모리 셀을 얼마나 유지할지 결정하며, 기억 게이트는 현재 시점의 새로운 정보를 얼마나 받아들일지를 결정

출력 게이트



$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

- 메모리 셀이 계산됐다면, 어떤 정보를 출력할지 결정해야 함
- 출력 게이트는 현재 시점의 은닉 상태를 제어. 출력 게이트도 망각 게이트에서 사용된 수식과 동일한 수식을 사용하며, 활성화 함수로 시그모이드를 사용
- 시그모이드 함수는 입력값을 [0, 1] 범위로 변환하므로 현재 시점의 은닉 상태 값을 얼마나 사용할지 결정

은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

- 현재 시점의 은닉 상태 h_t 는 출력 게이트와 하이퍼볼릭 탄젠트를 적용한 메모리 셀 값으로 계산됨
- 출력 게이트는 [0, 1]의 범위를 가지며, 하이퍼볼릭 탄젠트가 적용된 메모리 셀은 [-1, 1] 범위를 가짐

- 두 값을 아다마르 곱 연산하면 현재 시점의 은닉 상태가 이전 시점의 은닉 상태에서 얼마나 영향을 받는지 계산할 수 있게 됨
- 이러한 방법으로 장단기 메모리는 현재 은닉 상태에서 이전 은닉 상태의 정보의 중요도를 제어할 수 있음

LSTM 클래스 파라미터 (PyTorch)

장단기 메모리도 파이토치에서 쉽게 구현할 수 있도록 장단기 메모리 클래스를 제공

파라미터	설명
<code>input_size</code>	입력 특성 크기
<code>hidden_size</code>	은닉 상태 크기
<code>num_layers</code>	장단기 메모리 층수
<code>bias</code>	편향 값 사용 유무
<code>batch_first</code>	입력 배치 크기를 첫 번째 차원으로 사용할지 여부
<code>dropout</code>	드롭아웃 확률
<code>bidirectional</code>	양방향 처리 여부
<code>proj_size</code>	선형 투사(Linear Projection) 크기 결정. 0보다 큰 경우, 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑. 이 값을 통해 출력 차원을 줄이거나 다른 차원으로 변환할 수 있음. 투사 크기가 0이라면 은닉 상태의 차원을 변환하지 않고 그대로 유지. 은닉 상태 크기(hidden_size)보다 작은 값으로 설정해야 함

장단기 메모리 클래스와 순환 신경망 클래스의 차이

- 장단기 메모리 클래스는 순환 신경망 클래스와 거의 유사한 구조를 가짐
- 장단기 메모리 클래스는 활성화 함수를 명확하게 정의해 사용하므로 순환 신경망에서 사용하는 **활성화 함수(nonlinearity) 매개변수를 사용하지 않음**
- **투사 크기(proj_size) 매개변수**를 사용해 장단기 메모리 계층의 출력에 대한 선형 투사(Linear Projection) 크기를 결정
 - 투사 크기가 0보다 큰 경우, 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑. 이 값을 통해 출력 차원을 줄이거나 다른 차원으로 변환할 수 있음
 - 투사 크기가 0이라면 은닉 상태의 차원을 변환하지 않고 그대로 유지
- 장단기 메모리 클래스는 출력 차원을 조절할 때 투사 크기 매개변수로 모델의 크기와 복잡도를 변경할 수 있음. 그러므로 투사 크기는 은닉 상태 크기(hidden_size)보다 작은 값으로 설정

- 메모리 셀의 상태도 입력으로 전달해야 하며, 은닉 상태와 메모리 셀 상태를 튜플로 묶어 사용
- 장단기 메모리는 순환 신경망과 비슷한 구조를 갖지만, 투사 크기가 적용되는 경우 차원 계산 방식이 달라짐. 또한 메모리 셀의 상태도 입력으로 전달해야 하며, 은닉 상태와 메모리 셀 상태를 튜플로 묶어 사용하므로 사용에 주의

입출력 차원

- 장단기 메모리 클래스는 순전파 연산 시 입력값(inputs)과 초기 은닉 상태(h_0), 초기 메모리 셀 상태(c_0)로 순방향 연산을 수행해 출력값(outputs)과 최종 은닉 상태(h_n), 최종 메모리 셀 상태(c_n)를 반환
- 입력값 차원은 앞선 순환 신경망 클래스에서 사용하던 입력값 차원 형태와 동일한 형태로 [배치 크기, 시퀀스 길이, 입력 특성 크기]를 전달
- 초기 은닉 상태도 순환 신경망 클래스와 동일한 구조를 갖지만, 장단기 메모리는 선형 투사를 통해 출력층의 차원을 변경할 수 있으므로 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력
- 초기 메모리 셀은 순환 신경망 클래스의 초기 은닉 상태와 동일한 구조인 [계층 수 × 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]를 가짐. 초기 메모리 셀은 선형 투사를 적용하더라도 은닉 상태 크기를 사용
- 순방향 연산 결과는 출력값과 최종 은닉 상태, 최종 메모리 셀 상태가 반환됨. 출력값은 수식도 순환 신경망과 동일한 수식을 사용하지만, 선형 투사가 적용됐으므로 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) × 투사 크기(또는 은닉 상태 크기)]로 구성됨
- 최종 은닉 상태는 초기 은닉 상태와 동일한 차원인 [3×2, 4, 256] 형태로 반환됨. 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의

5. 문장 분류 모델 실습 (RNN/LSTM)

모델 구조

- 이번 절에서는 순환 신경망과 장단기 메모리를 활용해 문장 긍/부정 분류 모델을 학습

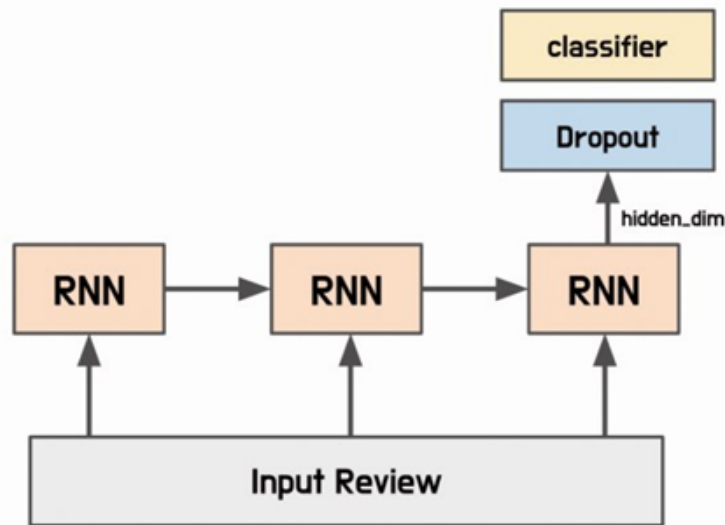


그림 6.23 긍/부정 분류 모델 구조

임베딩 계층

- 임베딩은 임베딩 벡터를 가져오는 임베딩 계층을 의미. 임베딩 계층은 [어휘 사전의 크기 × 임베딩 벡터의 크기]로 순람표 구조를 가짐
- 임베딩 계층은 입력 텍스트를 정수 인코딩한 뒤 해당 색인 값에 해당하는 임베딩을 가져오는 역할을 함
- 초깃값으로는 무작위 값을 할당하고, 학습을 통해 임베딩 계층이 최적화되게 만들거나, 사전 학습된 임베딩 벡터를 가져와 사용할 수 있음

SentenceClassifier 클래스

- `model_type` 매개변수로 순환 신경망을 사용할지, 장단기 메모리를 사용할지 설정
- 초기화 메서드에서는 SentenceClassifier 클래스에 입력된 함수의 매개변수에 따라 모델 구조를 미세조정. 분류기 계층은 모델을 양방향으로 구성한다면 전달되는 입력 채널 수가 달라지므로 분류기 계층을 현재 모델 구조에 맞게 변경 (`hidden_dim * 2`)
- 순방향 메서드에서는 입력받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값을 얻음. 그리고 얻은 임베딩 값을 모델에 입력하여 출력값을 얻음. 출력값(output)의 마지막 시점만 활용할 예정이므로 `[:, -1, :]` 으로 마지막 시점의 결괏값만 분리해 분류기 계층에 전달

손실 함수 및 최적화

BCEWithLogitsLoss

- 현재 모델은 문장의 긍/부정을 분류하므로 손실 함수는 이진 교차 엔트로피 함수를 적용
- BCEWithLogitsLoss 클래스는 BCELoss 클래스와 Sigmoid 클래스가 결합된 형태. BCELoss 클래스를 사용하는 경우, `criterion(torch.sigmoid(output), target)` 과 동일한 형태로 간주할 수 있음
- 교차 엔트로피는 내부적으로 소프트맥스 연산을 수행하므로 신경망의 출력값을 후처리 없이 활용할 수 있음

RMSProp (Root Mean Square Propagation)

- 최적화 함수는 RMSProp을 적용
- RMSProp은 모든 기울기를 누적하지 않고, **지수 가중 이동 평균(Exponentially Weighted Moving Average, EWMA)** 을 사용해 학습률을 조절
- 기울기 제공 값의 평균값이 작아지면 학습률을 증가시키고, 그 반대일 경우 학습률을 감소시켜서 불필요한 지역 최솟값에 빠지는 것을 방지
- RMSprop은 기울기의 크기가 큰 경우에는 빠른 수렴을 보이며, 작은 경우에는 더 작은 학습률을 유지함으로써 더 안정적으로 최적화를 수행할 수 있음

검증 지표

- 에폭마다 테스트 데이터셋으로 모델의 **검증 손실(Validation Loss)** 과 **검증 정확도(Validation Accuracy)** 를 확인
- 검증 손실과 검증 정확도는 모델이 이전에 본 적이 없는 테스트 데이터셋으로, 모델이 새로운 데이터를 얼마나 잘 예측하는지를 평가

사전 학습된 임베딩 초기화

- 학습된 모델의 임베딩 계층의 가중치를 동일한 단어 사전을 사용하는 토큰의 임베딩 값으로 사용할 수도 있음
- 이번에는 사전 학습된 임베딩 값을 초깃값으로 적용해 모델을 학습시키는 방법을 알아본다. 6.4 'Word2Vec' 절에서 동일한 말뭉치로 학습한 Word2Vec 모델로 임베딩 계층을 초기화
- 사전 학습된 모델로 임베딩 계층을 초기화하는 경우 넘파이 배열로 초기화할 수 있음. 단, 이때 `<pad>` 토큰과 `<unk>` 토큰은 초기화에서 제외
- 임베딩 계층은 파이토치의 임베딩 클래스의 `from_pretrained` 메서드로 초기화할 수 있음. 이 값을 SentenceClassifier 클래스의 `self.embedding` 값으로 적용
- 사전 학습된 임베딩을 사용하는 것은 모델 성능을 개선할 수 있는 방법 중 하나. 하지만 학습 데이터의 양이 충분히 많다면, 모델의 목적에 맞게 새로운 임베딩 층을 학습하는 것

이 더 좋은 결과를 얻을 수도 있음

- 또한, 사전 학습된 임베딩을 사용하더라도 해당 언어와 문제에 맞는 임베딩을 선택하는 것이 중요. 예를 들어, **한국어 자연어 처리에서는 문맥 정보를 고려한 임베딩 방법이 더 성능이 좋을 수 있음**. 따라서 모델의 목적과 데이터의 특성을 고려하여 적절한 임베딩 방법을 선택하는 것이 중요

6. 합성곱 신경망 (CNN)

개요

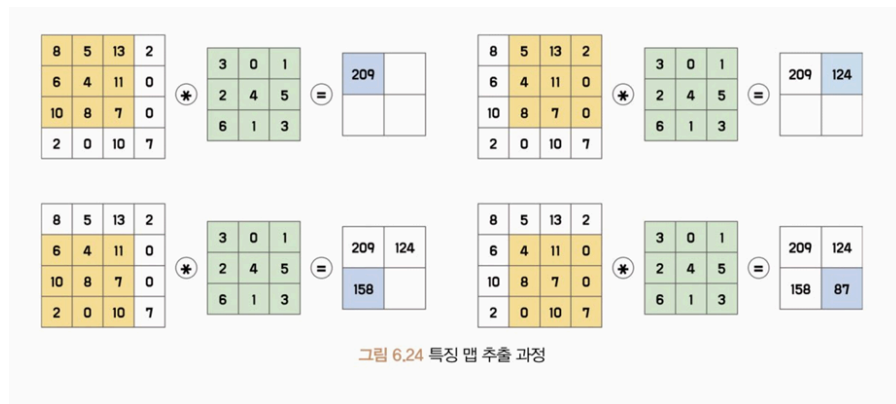
- **합성곱 신경망(Convolutional Neural Network, CNN)** 은 주로 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류
- 합성곱 신경망은 입력 데이터의 지역적인 특징을 추출하는 데 특화된 구조를 갖고 있으며 이를 위해 **합성곱(Convolution)** 연산을 사용
- 합성곱 연산은 이미지의 특정 영역에서 입력값의 분포 또는 변화량을 계산해 출력 노드를 생성. 특정 영역 안에서 연산을 수행하므로 **지역 특징(Local Features)** 을 효과적으로 추출할 수 있음
- 이미지 데이터는 고정된 프레임 내에 객체들의 위치와 형태가 자유분방하므로 여러 영역의 지역 특징을 조합해 입력 데이터의 전반적인 **전역 특징(Global Features)** 을 파악할 수 있음

자연어 처리에서의 CNN

- 합성곱 신경망은 원래 컴퓨터비전을 위해 고안된 인공 신경망이지만, 자연어 처리 작업에서도 우수한 성능을 보임
- 앞선 순환 신경망은 이전 시점($t-1$)의 상태를 기억해 현재 상태(t)를 계산하는 데 유용한 구조이지만, **연산 순서의 제약으로 병렬 처리가 어렵다**는 단점이 있음
- 입력 데이터의 길이가 길어질수록 처리 속도가 느려지고 많은 수의 가중치를 사용하면 학습이 어려워지는 문제가 있어 **깊은 신경망을 구성하기가 어려움**
- 2014년 사전 학습된 임베딩과 합성곱 신경망을 이용한 문장 분류 모델의 등장으로 자연어 처리에서도 합성곱 신경망이 사용되기 시작했음
- 입력 데이터의 길이에 상관없이 병렬 처리가 가능하고, 학습에 필요한 가중치 수를 줄여 깊은 신경망을 구성할 수 있게 됐음

합성곱 계층

- 합성곱 계층은 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층. 합성곱 계층은 이미지나 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용
- 합성곱 계층은 필터를 사용해 데이터의 특징을 추출하므로 데이터의 지역적인 패턴을 인식할 수 있으며, 입력 데이터의 모든 위치에서 동일한 필터를 사용하므로 **모델 매개변수를 공유**
- 모델의 매개변수를 공유함으로써 모델이 학습해야 할 매개변수 수가 감소해 **과대적합을 방지**
- 또한 입력 데이터에서 특징을 추출할 때, 해당 특징이 이미지 내 다른 위치에 존재하더라도 필터를 사용해 특징을 추출하므로 특징이 어디에 있어도 동일하게 추출할 수 있음
- 이러한 합성곱 계층을 여러 겹 쌓아 모델을 구성하며, **합성곱 계층이 많아질수록 모델의 복잡도가 증가**하므로 더 다양한 특징을 추출해 학습할 수 있음



합성곱 계층 출력 크기 계산

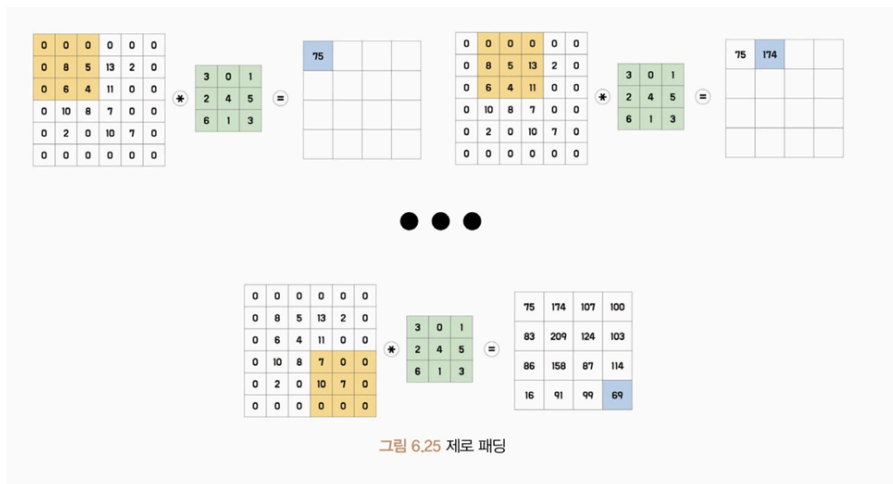
$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

필터 (Filter)

- 합성곱 계층은 입력 데이터에 **필터(Filter)** 를 이용해 합성곱 연산을 수행하는 계층. 필터는 **커널(Kernel)** 또는 **윈도우(Window)** 로 불리기도 하며, 일반적으로 3×3, 5×5와 같은 작은 크기의 정방형으로 구성됨
- 이 필터를 일정 간격으로 이동하면서 입력 데이터와 합성곱 연산을 수행해 특징 맵을 생성. 필터 영역마다 합성곱 연산이 수행되며, 필터의 가중치가 이 모델 학습 과정에서 갱신됨
- 합성곱 신경망은 보통 여러 개의 필터를 사용해 다양한 특징을 추출하며, 이를 통해 입력 이미지의 다양한 특징을 인식하고 분류할 수 있음

패딩 (Padding)

- 앞선 그림 6.24의 예시와 같이 일반적으로 합성곱 연산을 수행하면 출력값인 특징 맵의 크기가 작아짐. 즉, 입력값의 크기가 4×4 이었지만, 2×2 크기로 감소
- 이는 합성곱 신경망을 더 깊게 쌓는 데 제약사항이 될 수 있으며, 합성곱 신경망 성능에 악영향을 끼칠 수 있음. 또한, 이미지의 가장자리에 있는 정보는 다른 위치에 있는 정보에 비해 학습하기가 더 어려움
- 앞선 그림 6.24를 다시 살펴보면 (1, 1) 위치의 픽셀값 8은 특징 맵을 계산하기 위해 한 번만 수식에 포함되어 영향력이 낮으며, (2, 2) 위치의 픽셀값 4는 네 번의 연산에 관여. 이와 같이 가장자리 부분의 정보가 학습되는 데 제한이 있을 수 있음
- 이러한 현상을 방지하기 위해 입력 이미지나 입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙이는 **패딩(Padding)** 을 추가. 가장자리에 덧붙이는 패딩 값은 0으로 할당하는데, 이를 **제로 패딩(Zero padding)** 이라고 함



- 그림 6.25와 같이 패딩을 추가하면 출력값의 크기가 작아지지 않고 입력값 크기와 동일한 4×4 크기로 특징 맵이 계산됨
- 합성곱 신경망에서는 패딩을 적절히 사용해 입력 데이터의 크기를 조정하고, 작은 크기의 필터로도 이미지 전체에 대한 정보를 반영할 수 있음

간격 (Stride)

- **간격(Stride)** 이란 필터가 한 번에 움직이는 크기를 의미. 간격의 크기가 1이면 필터가 한 픽셀씩 이동하면서 합성곱 연산을 수행하며, 간격의 크기가 2 이상이면 필터가 더 큰 간격으로 이동하면서 계산을 수행
- 간격의 크기를 조절함으로써 출력 데이터의 크기를 조절할 수 있음. 예를 들어, 앞선 그림 6.25는 간격을 1로 설정해 필터가 한 픽셀씩 이동하면서 출력 데이터를 생성하므로 입력 데이터와 동일한 크기의 출력 데이터가 생성됨

- 간격의 크기를 2 이상으로 할당했다면 필터가 더 큰 간격으로 이동하므로 출력 데이터의 크기는 입력 데이터보다 작아짐
- 간격을 조절함으로써 입력 데이터의 공간적인 정보를 유지하거나 감소시킬 수 있음. 입력 데이터의 공간적인 정보는 픽셀 간의 상대적인 위치나 거리에 대한 정보를 의미
- 예를 들어, 이미지에서 픽셀 간의 상대적인 위치나 거리에 대한 정보는 이미지의 형태나 구조를 나타내는 데 중요한 역할을 함
- 간격을 작게 설정하면 입력 데이터의 공간적인 정보를 보존할 수 있으며, 간격을 크게 설정하면 입력 데이터의 공간적인 정보를 감소시킬 수 있음
- 간격을 조절해 합성곱 신경망이 학습해야 하는 모델 매개변수의 수를 감소시킬 수 있으며, 이를 통해 모델의 복잡도를 낮추고 과대적합을 방지할 수 있음

채널 (Channel)

- 입력 데이터와 필터 간의 연산은 **채널(Channel)** 에서 수행됨. 채널은 입력 데이터와 필터가 3차원으로 구성되어 있을 때 같은 위치의 값끼리 연산되게 함. 이를 통해 입력 데이터의 공간 정보를 유지하면서 추출되는 특징을 확장할 수 있음
- 예를 들어, 입력 데이터가 RGB 이미지라면, 각각의 R, G, B 채널마다 동일한 필터가 존재하며, 각 필터는 해당 채널의 정보를 추출해 특징 맵을 생성. 특징 맵의 개수는 채널의 개수만큼 존재. 만약 그림 6.25의 입력이 RGB 이미지였다면, 하나의 채널에 대해 연산한 결과가 됨
- 채널 개수는 일반적으로 합성곱 계층에서 설정되며, 이는 모델의 구조나 목적에 따라 달라짐. 채널의 개수가 많아질수록 학습할 수 있는 특징의 다양성이 증가해 모델의 **표현력 (Representational Power)** 이 높아지는 효과를 가져옴
- 출력 채널이 많은 경우, 각 채널은 입력 데이터에서 서로 다른 특징을 학습할 수 있음. 따라서 모델은 더 많은 종류의 특징을 학습하게 되며, 그로 인해 더 복잡한 문제를 해결할 수 있는 능력을 갖추게 됨
- 그러나 출력 채널이 많을수록 모델의 매개변수가 많아지므로 학습 시간과 메모리 사용량이 증가하는 단점을 갖게 됨. 그러므로 모델이 과대적합 되는 위험이 발생

팽창 (Dilation)

- **팽창(Dilation)** 이란 합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법. 팽창은 필터와 입력 데이터 사이에 간격을 두는 방법
- 일반적으로 합성곱 계층에서 팽창값은 1로 사용해 간격을 한 칸만 띄워 사용. 이 값이 커질수록 필터가 입력 데이터를 바라보는 범위가 넓어짐

- 팽창은 필터의 크기를 키우지 않고 입력 데이터에 더 넓은 영역을 고려할 수 있게 해 더 깊고 복잡한 모델을 구성할 수 있게 함. 팽창을 적절히 적용하면 모델의 매개변수 수를 줄일 수 있어 메모리 사용량을 줄일 수 있음
- 하지만 필터가 바라봐야 하는 입력 데이터의 범위가 커지므로 오히려 연산량이 늘어날 수도 있음. 또한 팽창 크기가 너무 크다면 인접한 픽셀값을 고려하지 않게 되므로 **공간적인 정보가 보존되지 않아 특징 추출의 효과가 떨어질 수 있음**

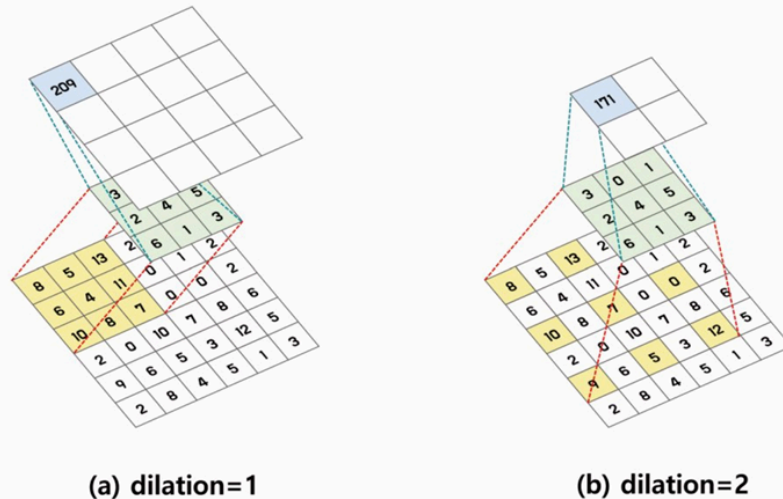


그림 6.26 팽창

활성화 맵 (Activation Map)

- **활성화 맵(Activation Map)** 은 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값을 의미. 합성곱 계층에서 입력 이미지와 필터의 합성곱 연산을 통해 특징 맵을 추출하면 이 값에 비선형성을 추가하기 위해 활성화 함수를 적용
- 일반적으로 합성곱 신경망에서는 **ReLU 함수**가 적용되며, 이를 통해 특징 맵의 값이 0 보다 크면 그 값을 그대로 출력하고, 0 이하일 경우에는 0을 출력
- 이렇게 얻은 활성화 맵은 다음 계층의 입력값으로 사용됨. 다음 계층이 합성곱 계층이라면 동일한 방식의 연산이 수행돼 합성곱 연산과 활성화 함수를 거치게 됨
- 합성곱 연산과 활성화 함수를 여러 번 반복하여 신경망을 구성하면, 입력 이미지에서 추출된 추상적인 특징을 학습할 수 있게 됨
- 합성곱 계층의 출력값에 활성화 함수를 적용하지 않으면 합성곱 연산 결과값이 그대로 다음 계층으로 전달됨. 이 경우 모델이 선형적인 결합만 수행하게 되어 **복잡한 패턴이나 추상적인 특징을 학습하는 것이 어려워짐**
- 따라서 활성화 함수를 적용함으로써 모델이 비선형성을 가지게 되어 입력 데이터에서 다양한 추상적인 특징을 학습할 수 있게 됨

- 활성화 맵은 합성곱 모델을 분석하는 데 매우 중요한 정보를 제공. 활성화 맵을 시각화하여 확인하면 **입력 이미지에서 모델이 어떤 특징을 학습하는지, 어떤 부분이 활성화되어 있는지** 등을 이해할 수 있음

풀링 (Pooling)

- **풀링(Pooling)** 은 특징 맵의 크기를 줄이는 연산으로 합성곱 계층 다음에 적용됨. 풀링은 특징 맵의 크기를 줄여 연산량을 감소시키고 입력 데이터의 정보를 압축하는 효과를 가짐
- 풀링은 합성곱 연산과 비슷하게 필터와 간격을 이용. 일정한 크기의 필터 내 특징 값을 선택. 선택하는 방법에는 크게 **최댓값 풀링(Max Pooling)** 과 **평균값 풀링(Average Pooling)** 이 있음
- 최댓값 풀링은 특정 크기의 필터 내 원소값 중 가장 큰 값을 선택해 특징 맵의 크기를 감소시키며, 평균값 풀링은 필터 내 원소값의 평균값으로 특징 맵의 크기를 감소시킴

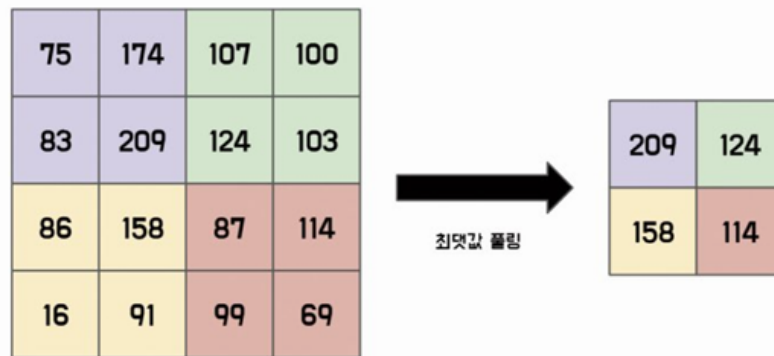


그림 6.27 최댓값 풀링

- 풀링은 입력 데이터의 공간적 크기를 줄이기 때문에 계산 비용을 감소시킬 수 있으며, 입력 데이터의 특징 위치가 변경되더라도 인근 영역에 대한 연산을 적용하기 때문에 공간적 정보를 유지할 수 있음
- 하지만 입력 데이터의 위치 정보를 일부 손실시키기 때문에 세밀한 위치 정보가 필요한 작업에서는 성능 저하를 초래할 수 있음
- 이로 인해 최근에는 풀링을 이용해 공간적 크기를 감소하는 방법보다 연산량이 더 많더라도 **합성곱 계층의 간격을 설정해 입력 데이터의 공간적인 크기를 줄이는 방법**을 사용

평균값 풀링 관련 추가 사항

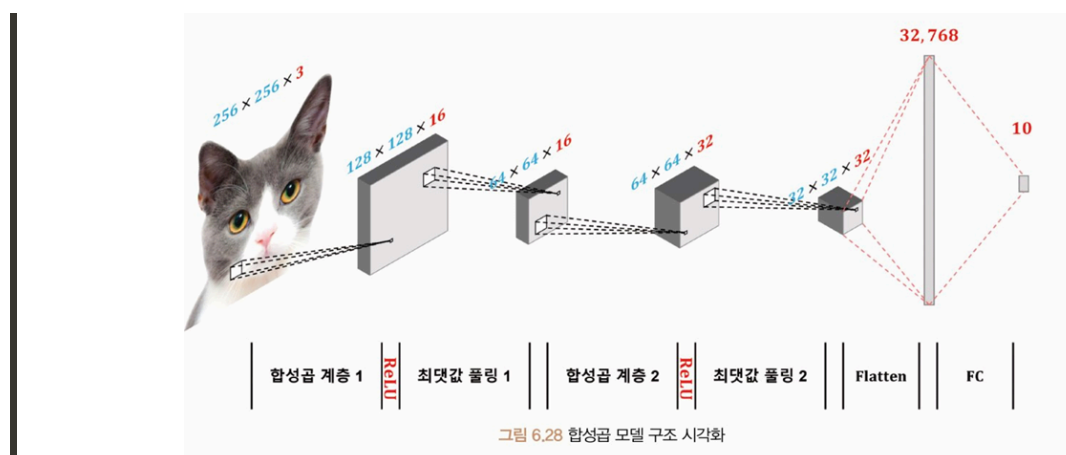
- 평균값 풀링 클래스도 합성곱 계층 클래스에서 사용했던 매개변수와 거의 동일. 단, 평균값으로 풀링해야 하므로 **팽창은 지원하지 않음. 팽창의 경우 원소 간의 거리가 멀어지므**

로 주변 영역의 특징 계산이 어려워지기 때문

- `count_include_pad`: 패딩 영역의 값을 평균 계산에 포함할지 여부를 설정. 참값으로 사용하면 **제로 패딩의 값이 평균값 풀링 연산에 포함됨**

완전 연결 계층 (Fully Connected Layer, FC)

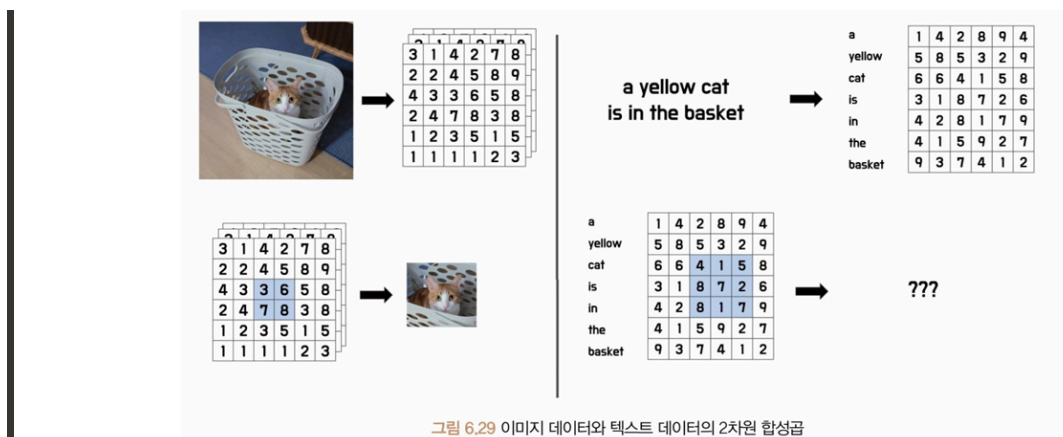
- **완전 연결 계층(Fully Connected Layer, FC)** 은 각 입력 노드가 모든 출력 노드와 연결된 상태를 의미. 이를 통해 완전 연결 계층은 입력과 출력 간의 모든 가능한 관계를 학습할 수 있음
- 이 계층은 출력 노드의 수를 조절할 수 있으므로 모델의 복잡성과 용량을 조절하는 데 사용됨. 예를 들어 이전 계층의 출력 데이터가 2차원 배열 형태인 경우 1차원 벡터 형태로 변경해 출력할 수 있음
- 일반적으로 합성곱 신경망에서는 합성곱 계층과 풀링 계층을 거친 결과물인 특징 맵을 입력으로 받음. 입력된 3차원 특징 맵은 평탄화 작업을 통해 1차원 벡터로 변경하고 완전 연결 계층의 가중치와 내적 연산을 수행해 출력값을 계산
- 전체 입력 특징 맵과 가중치 간의 내적 연산을 수행해 출력값을 계산하므로 이전 계층에서 추출한 특징 맵의 **공간 정보가 무시되고 모든 입력을 독립적으로 처리**해 계산
- 합성곱 신경망에서는 특징 맵의 공간 정보를 보존하기 위해 합성곱 계층으로 구성된 네트워크를 구성하고, 이후에 완전 연결 계층을 추가하여 고수준 작업을 수행
- 완전 연결 계층은 이전 계층에서 추출된 특징을 활용하여 최종적인 분류 작업을 수행. 최종값은 소프트맥스나 시그모이드와 같은 활성화 함수를 적용해 분류 모델로 구성할 수 있음
- 따라서 완전 연결 계층은 합성곱 신경망의 최종 출력을 결정하는 역할을 함



합성곱 계층 클래스 파라미터 (PyTorch `nn.Conv2d`)

파라미터	설명
<code>in_channels</code>	입력 데이터 채널 크기(in_channels)와 출력 데이터 채널 크기(out_channels)를 통해 채널을 생성. 앞선 선형 변환 클래스의 입력 데이터 차원 크기(in_features)와 출력 데이터 차원 크기(out_features) 매개변수와 동일한 역할을 함
<code>out_channels</code>	출력 데이터 채널 크기
<code>kernel_size</code>	합성곱 연산의 필터 크기를 의미
<code>stride</code>	간격(stride), 패딩(padding), 팽창(dilation)은 앞서 설명한 필터의 속성을 의미
<code>padding</code>	패딩 크기 (기본값 0)
<code>dilation</code>	팽창 크기 (기본값 1)
<code>groups</code>	입력 채널과 출력 채널을 하나의 그룹으로 묶는 것을 의미. 그룹을 2 이상의 값으로 설정하면 입력 채널과 출력 채널을 더 작은 그룹으로 나눠 각각 합성곱 연산을 수행. 그룹을 사용하면 합성곱 연산 시 모델의 매개변수 수를 줄일 수 있음. 그룹의 값이 2 이상이면 입력 채널이나 출력 채널이 그룹 수의 배수여야 함
<code>bias</code>	편향 설정(bias)은 계층에 편향 값 포함 여부를 설정
<code>padding_mode</code>	패딩 모드(padding_mode)는 패딩 영역에 할당되는 값을 설정. zeros는 제로 패딩을 의미하며, reflect는 입력 데이터의 가장자리를 거울처럼 반사해 값을 할당. replicate는 가장자리의 값을 복사해 값을 할당

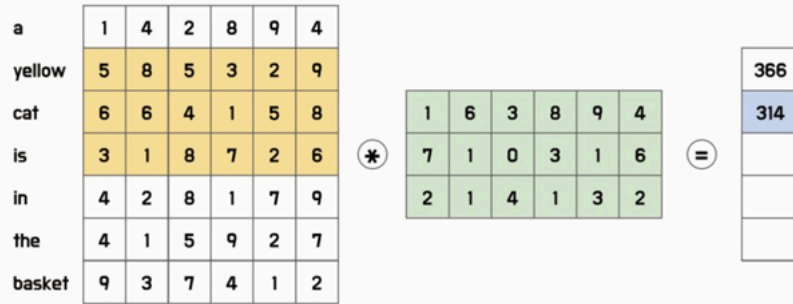
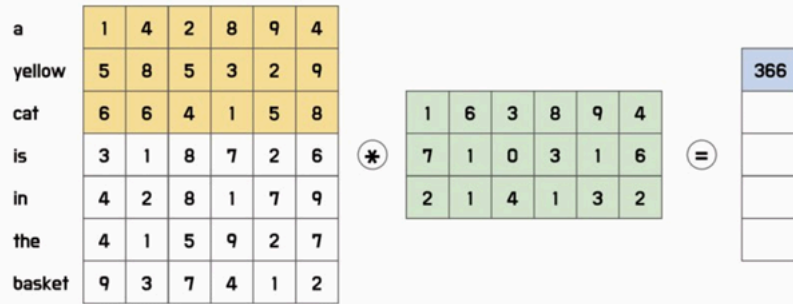
7. CNN 기반 문장 분류 모델 실습



텍스트에서의 CNN

- 이번 절에서는 합성곱 신경망을 활용해 자연어 처리 모델을 구성해 본다

- 앞선 설명에서는 수직/수평 방향으로 이동하며, 합성곱 연산을 수행하는 **2차원 합성곱 (2-Dimensional Convolution)**에 대해 알아봤음
- 그러나 텍스트 데이터의 임베딩 값은 입력 순서를 제외하면 입력값의 위치가 의미를 가지지 않음. 그러므로 텍스트 데이터에서도 이미지 데이터와 같이 2차원 합성곱 필터를 사용하면 텍스트의 정보를 제대로 학습할 수 없음
- 텍스트 데이터에는 **1차원 합성곱(1-Dimensional Convolution)**을 적용해야 함. 1차원 합성곱은 입력 데이터가 1차원 벡터인 경우에 대한 합성곱 연산을 수행
- 1차원 합성곱은 일반적으로 텍스트 데이터 처리에서 많이 사용됨. 텍스트 데이터는 문장을 단어 단위로 분리하여 각 단어를 임베딩하여 나온 1차원 벡터 데이터를 입력값으로 사용
- 1차원 합성곱은 이러한 입력값에서 수평 방향으로 이동하지 않고 **수직 방향으로만 이동하는 필터**를 적용하여 해당 문장의 특징을 추출
- 1차원 필터의 크기는 필터의 높이에만 영향을 미치며, 필터의 너비는 입력 임베딩의 크기가 됨
- 필터 크기가 3인 합성곱 임베딩을 수행하면 인접한 3개의 토큰에 대해 연산을 수행. 이는 앞서 다룬 **N-gram과 유사한 개념**으로, 텍스트 순서에 따른 정보를 학습할 수 있음
- 또한, 1차원 합성곱을 이용한 신경망에서는 다양한 크기의 합성곱 필터를 사용하여 여러 종류의 정보를 추출할 수 있음. 1차원 합성곱을 수행하면 1차원 벡터를 출력값으로 얻게 되므로 풀링을 적용하면 하나의 스칼라값이 도출됨
- 그러므로 크기가 다른 여러 개의 합성곱 필터를 사용하면 여러 개의 스칼라값을 얻을 수 있고 이러한 다양한 크기의 출력 벡터를 모아 하나의 벡터로 연결하여 하나의 특징 벡터로 만들 수 있음



⋮

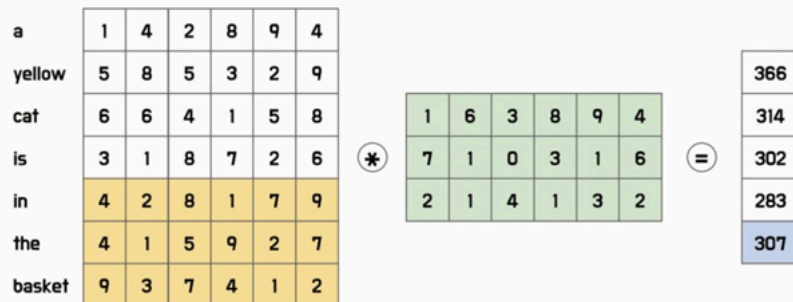


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱



모델 구조 (SentenceClassifier)

- SentenceClassifier 클래스는 사전 학습된 임베딩 벡터(pretrained_embedding), 필터 크기 리스트(filter_sizes), 최대 길이(max_length)를 입력받아 모델을 구성
- 합성곱 계층 구성 시 그림 6.31과 같이 크기가 다른 여러 개의 합성곱 필터를 구성하기 위해 **필터 크기 리스트만큼 시퀀셜을 생성**
- 각 시퀀셜은 1차원 합성곱 계층, ReLU, 최댓값 풀링으로 구성됨. 합성곱 계층의 출력 채널 수는 1로 설정. 최댓값 풀링의 커널 크기는 앞선 합성곱 연산의 영향을 받으므로 **최대 길이 - 필터 크기 - 1**로 설정
- **ModuleList** 클래스는 여러 개의 서브 모듈을 리스트 형태로 저장하는 기능을 제공. conv 리스트에는 시퀀스 모듈이 담겨 있으므로 ModuleList 클래스를 적용
- 순방향 메서드에서는 입력값(inputs)을 임베딩 계층에 통과시켜 임베딩 벡터를 얻음. 이후 permute 메서드를 통해 차원을 변경하고 앞서 설정한 여러 개의 합성곱 필터에 통과 시킴
- **cat 메서드**를 통해 각 필터의 출력값을 이어 붙여 하나의 벡터로 만들고 분류를 위한 네트워크를 통과시켜 최종 출력값을 얻음

손실 함수 및 최적화

- 손실 함수: **BCEWithLogitsLoss**
- 최적화: **Adam(Adaptive Moment Estimation)** (lr=0.001)
 - RMSProp이 아닌 Adam 사용
 - Adam은 경사 하강법 알고리즘을 개선한 방법으로 **모멘텀과 RMSProp을 결합한 방법**

- Adam은 이전 기울기 값의 지수 이동 평균과 이전 기울기 값의 지수 이동 제곱 평균을 사용해 현재 기울기 값을 갱신
- 모멘텀과 마찬가지로 지난 기울기 값의 영향을 일정 부분 유지하면서 새로운 기울기 값을 반영하기 때문에 기울기가 빠르게 변화하는 구간에서 더 안정적인 수렴을 보임
- 또한, RMSProp과 같이 학습률을 조절해 발산하는 경우를 방지
- Adam은 학습률, 모멘텀, RMSProp의 하이퍼파라미터를 모두 자동으로 조절하면서 최적의 값으로 수렴. 따라서 하이퍼파라미터를 조정하는 수고를 덜어줄 뿐 아니라, 학습률과 모멘텀의 조절 문제를 해결하여 SGD보다 더 빠른 수렴을 보이는 경우가 많음

CNN 기반 문장 분류의 특징 및 한계

- 텍스트 데이터를 합성곱 신경망으로 처리하면, 각 단어를 지역적인 특징으로 인식할 수 있어서 **문장 내부의 구조적인 정보를 잘 파악**할 수 있음
- 또한 합성곱 신경망은 계산 속도가 빠르기 때문에 **대규모 데이터셋에서도 빠르게 학습**할 수 있음
- 합성곱 신경망은 이미지와 유사한 구조를 가진 2차원 데이터에 더 적합하므로 **텍스트 데이터를 합성곱 신경망으로 처리한다면 모델의 구조 및 목적을 충분히 고려**해야 함